

**DETEKSI KANKER PARU PARU ADENOKARSINOMA DENGAN
ALGORITMA *RECURRENT NEURAL NETWORK* (RNN)
MENGUNAKAN DATA BIOMARKER DNA**

TUGAS AKHIR

Diajukan untuk melengkapi tugas-tugas
dan memenuhi syarat-syarat guna memperoleh gelar Sarjana Ilmu Komputer

Oleh:

RENDIKA RAHMATURRIZKI
2108107010066



**PROGRAM STUDI INFORMATIKA DEPARTEMEN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA, BANDA ACEH
JUNI, 2026**

PENGESAHAN

DETEKSI KANKER PARU PARU ADENOKARSINOMA DENGAN ALGORITMA *RECURRENT NEURAL NETWORK* (RNN) MENGGUNAKAN DATA BIOMARKER DNA

Oleh:

Nama : Rendika Rahmaturrizki
NPM : 2108107010066
Jurusan : Informatika

Menyetujui:

Pembimbing I

Pembimbing II

Dr. Muhammad Subianto, S.Si, M.Si.
NIP. 196812111994031005

Ir.Essy Harnelly, S.Si., M.Si., Ph.D.
NIP. 197501092000122002

Mengetahui:

Dekan FMIPA
Universitas Syiah Kuala,

Koordinator Prodi S1 Informatika FMIPA
Universitas Syiah Kuala,

Prof.Dr.Taufik Fuadi Abidin, S.Si., M.Tech
NIP. 197010081994031002

Rasudin S.Si., M.Info.Tech.
NIP. 197410011999031001

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini,

Nama lengkap : Rendika Rahmaturrizki
Tempat/tanggal lahir : Jakarta/ 24 Mei 2002
NPM : 2108107010066
Program Studi : Informatika
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Judul Tugas Akhir : DETEKSI KANKER PARU PARU
ADENOKARSINOMA DENGAN ALGORITMA
RECURRENT NEURAL NETWORK (RNN)
MENGGUNAKAN DATA BIOMARKER DNA

Menyatakan dengan sesungguhnya bahwa Laporan Tugas Akhir saya dengan judul di atas adalah **hasil karya saya sendiri** bersama dosen pembimbing dan **bebas plagiasi**.

Jika ternyata di kemudian hari terbukti bahwa Laporan Tugas Akhir merupakan hasil plagiasi, saya bersedia menerima sanksi yang berlaku di Universitas Syiah Kuala.

Banda Aceh, 16 Juni 2026

Rendika Rahmaturrizki
NPM. 2108107010066

SURAT PERNYATAAN

Yang bertanda tangan di bawah ini,

1. Nama : Rendika Rahmaturrizki
NPM : 2108107010066
Departemen/Prodi : Informatika
Status : Mahasiswa
2. Nama : Dr. Muhammad Subianto, S.Si, M.Si.
NIP : 196812111994031005
Departemen/Prodi : Informatika
Status : Pembimbing I
3. Nama : Ir.Essy Harnelly, S.Si., M.Si., Ph.D.
NIP : 197501092000122002
Departemen/Prodi : Informatika
Status : Pembimbing II

Dengan ini menyatakan hasil penelitian Tugas Akhir yang berjudul **DETEKSI KANKER PARU PARU ADENOKARSINOMA DENGAN ALGORITMA RECURRENT NEURAL NETWORK (RNN) MENGGUNAKAN DATA BIOMARKER DNA** tidak dipublikasikan secara *full-text* di sistem ETD (*Electronic Theses and Dissertations*) Universitas Syiah Kuala hingga batas waktu 5 tahun dari tanggal kelulusan.

Demikian surat pernyataan ini dibuat dengan sebenarnya untuk dapat dipergunakan seperlunya.

Darussalam, 16 Juni 2026
Yang membuat pernyataan,

Pembimbing I,

Pembimbing II,

Mahasiswa,

Dr. Muhammad Subianto, S.Si, M.Si.
NIP. 196812111994031005

Ir.Essy Harnelly, S.Si., M.Si., Ph.D.
NIP. 197501092000122002

Rendika Rahmaturrizki
NPM. 2108107010066

Mengetahui:

Koordinator Program Studi Informatika
Universitas Syiah Kuala,

Koordinator TA,

Rasudin S.Si., M.Info.Tech.
NIP. 197410011999031001

Sri Azizah Nazhifah, S.Kom., M.Sc
NIP. 199304072024062003

ABSTRAK

Kanker paru-paru adenokarsinoma merupakan salah satu penyebab kematian tertinggi yang arah terapi targetnya sangat bergantung pada deteksi spesifik mutasi biomarker DNA. Penelitian ini bertujuan untuk membangun dan membandingkan performa model *deep learning* berbasis *Recurrent Neural Network* (RNN), yaitu *Bidirectional Long Short Term Memory* (BiLSTM) dan *Bidirectional Gated Recurrent Unit* (BiGRU), dalam mendeteksi dan mengklasifikasikan sekuens DNA normal dan mutasi kanker paru-paru. *Dataset* sekunder berfokus pada lima gen target klinis utama (*EGFR*, *PD-L1*, *ALK*, *BRAF*, dan *ROS1*) yang diproses melalui teknik pra-pemrosesan dan augmentasi substitusi kodon sinonim dalam dua skenario: Skenario 1 (augmentasi hingga 500 sekuens per gen) dan Skenario 2 (batas 100 sekuens per gen). Evaluasi komputasi dilakukan menggunakan metrik *accuracy*, *precision*, *recall*, *F1-Score*, dan AUC, beserta uji inferensi menggunakan data eksternal. Hasil eksperimen menunjukkan bahwa arsitektur BiGRU pada Skenario 1 mencapai performa tertinggi dan paling stabil dengan tingkat akurasi 95,00%, *F1-Score* 0,9667, nilai presisi mutlak 1,0000, dan keberhasilan menekan *False Negative* hingga nol. Sementara itu, BiLSTM pada skenario yang sama mencapai akurasi 93,33%. Pada kondisi ketersediaan data minim (Skenario 2), kedua model mengalami anomali penurunan performa ekstrem berupa *overfitting* dan kebutaan kelas (*Class Collapse*). Penelitian ini membuktikan bahwa fusi keputusan jaringan saraf berarah ganda yang didukung oleh kelimpahan variansi data optimal terbukti tangguh, presisi, dan dapat diandalkan sebagai instrumen penapisan komputasi medis untuk onkologi molekuler.

Kata kunci: kanker paru-paru adenokarsinoma, biomarker DNA, *deep learning*, *Recurrent Neural Network*, BiLSTM, BiGRU, augmentasi data.

ABSTRACT

Lung adenocarcinoma is a leading cause of cancer-related mortality, where targeted therapy heavily relies on the specific detection of DNA biomarker mutations. This study aims to build and compare the performance of *deep learning* models based on *Recurrent Neural Network* (RNN) architectures, specifically *Bidirectional Long Short Term Memory* (BiLSTM) and *Bidirectional Gated Recurrent Unit* (BiGRU), in detecting and classifying normal and mutated lung cancer DNA sequences. The secondary *dataset* focused on five primary clinical target genes (*EGFR*, *PD-L1*, *ALK*, *BRAF*, and *ROS1*), which were processed using preprocessing techniques and synonymous codon substitution augmentation under two scenarios: Scenario 1 (augmentation up to 500 sequences per gene) and Scenario 2 (limited to 100 sequences per gene). Computational evaluation was conducted using *accuracy*, *precision*, *recall*, *F1-Score*, and AUC metrics, alongside an external inference test. The experimental results demonstrated that the BiGRU architecture in Scenario 1 achieved the highest and most stable performance with an accuracy of 95.00%, an *F1-Score* of 0.9667, an absolute precision of 1.0000, and successfully suppressed *False Negatives* to zero. Meanwhile, BiLSTM in the same scenario achieved 93.33% accuracy. Under limited data conditions (Scenario 2), both models suffered extreme performance anomalies, namely *overfitting* and *Class Collapse*. This research proves that bidirectional neural network decision fusion, supported by an optimal abundance of data variance, is robust, precise, and reliable as a computational medical screening instrument for molecular oncology.

Keywords: lung adenocarcinoma cancer, DNA biomarkers, *deep learning*, *Recurrent Neural Network*, BiLSTM, BiGRU, data augmentation.

KATA PENGANTAR

Segala puji dan syukur penulis panjatkan ke hadirat Allah SWT atas limpahan rahmat dan hidayah-Nya, sehingga penulis dapat menyelesaikan penulisan tugas akhir yang berjudul "Deteksi Kanker Paru-Paru Adenokarsinoma dengan Algoritma *Reccurent Neural Network* (RNN) menggunakan Data Biomarker DNA" dengan baik dan sesuai rencana. Penulis menyadari bahwa pencapaian ini tidak lepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan rasa terima kasih yang sebesar-besarnya kepada:

1. Monalisa dan Alm. Yulizar, selaku orang tua penulis, yang selalu memberikan dukungan penuh dalam setiap aktivitas dan kegiatan penulis, baik secara moral maupun material, serta menjadi motivasi utama dalam menyelesaikan tugas akhir ini.
2. Prof. Dr. Taufik Fuadi Abidin, S.Si., M.Tech., selaku Dekan FMIPA Universitas Syiah Kuala.
3. Dr. Nizamuddin, M.Info.Sc., selaku Ketua Departemen Informatika FMIPA Universitas Syiah Kuala.
4. Bapak Dr. Muhammad Subianto, S.Si., M.Si., selaku Dosen Pembimbing I, atas bimbingan, arahan, serta ilmu yang sangat berharga selama proses penyusunan tugas akhir ini.
5. Ibu Ir. Essy Harnelly, S.Si., M.Si., Ph.D., selaku Dosen Pembimbing II, yang telah memberikan bimbingan, dorongan, arahan, serta masukan yang sangat penting hingga tugas akhir ini dapat terselesaikan.
6. Bapak Rasudin, S.Si., M.Info.Tech., selaku Dosen Wali, atas dukungan dan pedoman yang diberikan selama penulis menempuh pendidikan.
7. Muhammad Akbarul Ihsan, Diky Wahyudi, Fajry Ariansyah, Furqan Al-Ghifari, dan Muhammad Firdaus, atas dukungan yang luar biasa selama empat tahun masa studi di Departemen Informatika USK dan selama penulisan skripsi.
8. Seluruh teman-teman seperjuangan dari Departemen Informatika USK angkatan 2021, atas dukungan moral yang luar biasa selama penulis menempuh pendidikan.
9. Seluruh Dosen dan Staf Departemen Informatika FMIPA USK, atas ilmu, bimbingan, dan pelayanan yang telah diberikan selama masa studi penulis.

Penulis menyadari bahwa dalam penulisan tugas akhir ini masih terdapat berbagai kekurangan, baik dari sisi materi, metode, maupun penggunaan bahasa. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun dari para pembaca guna meningkatkan kualitas karya ini. Besar harapan penulis agar tugas akhir ini dapat memberikan manfaat yang luas serta berkontribusi terhadap perkembangan ilmu pengetahuan.

Banda Aceh, 16 Juni 2026

Rendika Rahmaturrizki
NPM. 2108107010066

DAFTAR ISI

	<i>Halaman</i>
Halaman Judul.....	i
Halaman Pengesahan	ii
Pernyataan Bebas Plagiasi	ii
Surat Pernyataan	iii
Abstrak/Abstract	iv
Kata Pengantar	vi
Daftar Isi	ix
Daftar Tabel	xi
Daftar Gambar.....	xii
Daftar Lampiran.....	xiii
DAFTAR SINGKATAN.....	xiv
 BAB I PENDAHULUAN.....	 1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Manfaat Penelitian	4
 BAB II TINJAUAN KEPUSTAKAAN.....	 5
2.1 Dogma Sentral Biologi Molekuler	5
2.1.1 Biomarker	6
2.1.2 Biomarker Genomik pada Adenokarsinoma Paru-Paru	6
2.2 Bahasa Pemrograman dan Pustaka Pendukung.....	8
2.2.1 Python	8
2.2.2 BioPython	9
2.3 Algoritma dan Pemodelan pada Data Sekuensial	9
2.3.1 BLAST dari NCBI.....	9
2.3.2 MAFFT	9
2.3.3 <i>Deep Learning</i>	10
2.3.4 RNN.....	10
2.3.5 BiLSTM.....	12
2.3.6 BiGRU.....	14
2.3.7 TensorFlow	15
2.4 Penelitian Terkait	16
 BAB III METODOLOGI PENELITIAN	 18
3.1 Waktu dan Lokasi Penelitian.....	18
3.2 Jadwal Pelaksanaan	18
3.3 Alat dan Bahan	18
3.3.1 Sumber Dataset.....	18
3.3.2 Perangkat Keras	19
3.3.3 Perangkat Lunak	19
3.4 Metode Penelitian	19

3.4.1	Studi Literatur	20
3.4.2	Pengumpulan Data	21
3.4.3	Prapemrosesan Data.....	22
3.4.3.1	Penyejajaran dengan MAFFT	22
3.4.3.2	Pembagian Dataset	23
3.4.3.3	Augmentasi Data	23
3.4.3.4	Tokenisasi	23
3.4.3.5	Penyamaan Panjang (<i>Padding</i>)	24
3.4.4	Perancangan dan Pelatihan Model.....	24
3.4.5	Analisis Hasil Pengujian Model.....	25
3.4.6	Penulisan Skripsi.....	25
BAB IV	HASIL DAN PEMBAHASAN.....	26
4.1	Evaluasi Model.....	26
4.1.1	Metrik Evaluasi	26
4.1.2	Visualisasi Hasil Pelatihan.....	30
4.2	Analisis Hasil	38
4.2.1	Analisis Metrik Evaluasi Klasifikasi	38
4.2.2	Analisis Visualisasi Hasil Pelatihan dan Kinerja Model....	39
4.2.3	Analisis Validasi Eksternal	41
BAB V	KESIMPULAN DAN SARAN.....	44
5.1	Kesimpulan	44
5.2	Saran.....	44
DAFTAR PUSTAKA		46
LAMPIRAN.....		50

DAFTAR TABEL

	<i>Halaman</i>
Tabel 2.1 Penelitian Terkait.....	16
Tabel 3.1 Jadwal pelaksanaan penelitian	18
Tabel 4.1 Perbandingan Akurasi Model BiLSTM dan BiGRU	27
Tabel 4.2 Perbandingan Presisi Model BiLSTM dan BiGRU per Kelas ...	28
Tabel 4.3 Perbandingan <i>Recall</i> Model BiLSTM dan BiGRU per Kelas ...	28
Tabel 4.4 Perbandingan <i>F1-Score</i> Model BiLSTM dan BiGRU	29
Tabel 4.5 Perbandingan AUC Model BiLSTM dan BiGRU	30

DAFTAR GAMBAR

	<i>Halaman</i>
Gambar 2.1 Ilustrasi Proses Ekspresi Gen.....	5
Gambar 2.2 Arsitektur RNN	11
Gambar 2.3 Arsitektur BiLSTM	13
Gambar 2.4 Arsitektur GRU	14
Gambar 2.5 Arsitektur BiGRU	15
Gambar 3.1 Diagram Alir Penelitian	20
Gambar 3.2 Struktur Data Mentah	22
Gambar 3.3 Alur Prapemrosesan	22
Gambar 4.1 <i>Learning Curve</i> , BiLSTM skenario pertama	31
Gambar 4.2 <i>Learning Curve</i> , BiGRU skenario pertama	31
Gambar 4.3 <i>Learning Curve</i> , BiLSTM skenario kedua	32
Gambar 4.4 <i>Learning Curve</i> , BiGRU skenario kedua	32
Gambar 4.5 <i>Confusion Matrix</i> skenario pertama.....	34
Gambar 4.6 <i>Confusion Matrix</i> skenario pertama keseluruhan data.....	34
Gambar 4.7 <i>Confusion Matrix</i> skenario kedua	35
Gambar 4.8 <i>Confusion Matrix</i> skenario kedua keseluruhan data	36
Gambar 4.9 Hasil Inferensi Model pada skenario pertama	41
Gambar 4.10 Hasil Inferensi Model pada skenario kedua.....	42

DAFTAR LAMPIRAN

	<i>Halaman</i>
Lampiran 1. Tampilan Data Mentah	50
Lampiran 2. Kode <i>Import Library</i> dan pengecekan Letak Data	50
Lampiran 3. Kode <i>Parse</i> Data menjadi bentuk CSV	51
Lampiran 4. Kode Augmentasi dan Definisi Fungsi	52
Lampiran 5. Kode Implementasi Model BiLSTM & BiGRU	55
Lampiran 6. Kode Prediksi hasil Model.....	59
Lampiran 7. Biodata Penulis	62

DAFTAR SINGKATAN

NSCLC	: <i>Non-Small Cell Lung Cancer</i>
EGFR	: <i>Epidermal Growth Factor Receptor</i>
PCR	: <i>Polymerase Chain Reaction</i>
KRAS	: <i>Kirsten Rat Sarcoma</i>
ALK	: <i>Anaplastic Lymphoma Kinase</i>
BRAF	: <i>B-RAF Proto-Oncogene, Serine/Threonine Kinase</i>
ROS1	: <i>ROS Proto-Oncogene 1</i>
PD-L1	: <i>Programmed Death-Ligand 1</i>
ANN	: <i>Artificial Neural Network</i>
BiLSTM	: <i>Bidirectional Long Short Term Memory</i>
BiGRU	: <i>Bidirectional Gated Recurrent Unit</i>
RNN	: <i>Recurrent Neural Network</i>
MAFFT	: <i>Multiple Alignment using Fast Fourier Transform</i>
NCBI	: <i>National Center for Biotechnology Information</i>
MSL	: <i>Max Sequence Length</i>
AUC	: <i>Area Under Curve</i>
ROC	: <i>Receiver Operating Characteristic</i>
CT	: <i>Computed Tomography</i>
NGS	: <i>Next Generation Sequencing</i>
CNN	: <i>Convolutional Neural Network</i>
TKI	: <i>Tyrosine Kinase Inhibitor</i>
EML4	: <i>Echinoderm Microtubule-associated Protein-like 4</i>
BLAST	: <i>Basic Local Alignment Search Tool</i>
FFT	: <i>Fast Fourier Transform</i>
GPU	: <i>Graphics Processing Units</i>
TPU	: <i>Tensor Processing Units</i>

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Kanker paru-paru merupakan salah satu ancaman paling mematikan dalam onkologi modern, yang tercatat sebagai penyakit keganasan yang paling sering didiagnosis dalam skala global (12,4%) dan juga penyebab utama mortalitas akibat kanker dengan persentase mencapai 18,7% pada tahun 2022 (Bray et al., 2024). Secara klinis, patologi ini terbagi menjadi dua kategori utama, di mana *Non-Small Cell Lung Cancer* (NSCLC) merepresentasikan penyumbang kasus terbesar yang mencakup sekitar 85% dari total keseluruhan insiden. Di dalam spektrum NSCLC tersebut, penyakit ini diklasifikasikan lebih lanjut menjadi karsinoma sel skuamosa paru-paru sebesar 30% dan adenokarsinoma paru-paru yang mendominasi hingga 70% dari keseluruhan kasus (Rojiani & Rojiani, 2024). Adenokarsinoma tidak hanya menjadi subtype yang paling umum, tetapi frekuensi kemunculannya juga terus menunjukkan tren peningkatan yang signifikan. Tantangan paling krusial dari penanganan subtype ini terletak pada fase diagnostik, di mana tahap awal perkembangannya pada umumnya baru dapat diidentifikasi secara visual sebagai *ground glass nodules* melalui pencitraan makroskopis modern *computed tomography* (CT) (Succony et al., 2021). Ketergantungan pada alat penapisan radiologi ini memicu tingginya angka keterlambatan diagnosis, sebagaimana tercermin dari data di Indonesia yang menunjukkan bahwa lebih dari 70% kasus kanker paru-paru terdeteksi saat sudah mencapai tahap lanjut dan telah menyebar ke bagian tubuh lain (Putra et al., 2016). Mengingat adenokarsinoma sangat sering terjadi namun kerap terlambat ditangani melalui alat deteksi makroskopis, upaya diagnosis saat ini sangat bergantung pada penelusuran anomali di tingkat dasar pembentuk selnya, yaitu pada level genetik (Zhang et al., 2024).

Mutasi genetik memainkan peran penting dalam terjadinya kanker adenokarsinoma paru-paru, dengan mengetahui perubahan pada gen yang sering terdeteksi saat terjadi kanker (Chen et al., 2024). Mutasi ini menyebabkan disregulasi jalur sinyal seluler, yang mengakibatkan aktivitas sel tidak terkendali dan pembentukan tumor. Sebagai contoh, mutasi pada gen EGFR menyebabkan reseptor sinyal sel tetap aktif tanpa adanya sinyal eksternal sebagai *trigger*, memicu pertumbuhan sel yang berlebihan (Ladanyi & Pao, 2008). Secara biologis, deteksi kanker paru-paru melibatkan berbagai metode, seperti pencitraan dan biopsi jaringan, namun pengujian pada biomarker semakin dibutuhkan karena dapat mengidentifikasi perubahan genetik atau ekspresi protein spesifik yang terkait dengan kanker yang diteliti (Zhang et al.,

2024). Pengujian biomarker dilakukan pada *polymerase chain reaction* (PCR) dan *next generation sequencing* (NGS), yang penting untuk diagnosis akurat dan pengobatan yang dipersonalisasi pada adenokarsinoma paru-paru (Ettinger et al., 2021).

Kemajuan dalam ilmu biologi mengenai kanker telah memungkinkan identifikasi kanker paru-paru melalui biomarker, yang berfungsi sebagai indikator untuk mendeteksi kanker secara dini, sehingga memungkinkan penanganan lebih cepat dan efektif (S. Das et al., 2024). Pesatnya pertumbuhan data eksperimental dan klinis yang dihasilkan dari studi sistematis, peningkatan kompleksitas dalam bidang penanganan kesehatan, dan juga kebutuhan akan pengembangan model sistem telah menjadikan bioinformatika sebagai bidang yang menjadi perhatian bagi peneliti. Bioinformatika memainkan peran yang penting dalam mengintegrasikan berbagai jenis data untuk dapat mendukung pengembangan obat secara prediktif, preventif, dan terpersonalisasi (Yan, 2014). Dengan kemampuan analisis data genomik dalam skala besar, bioinformatika menjadi fondasi penting dalam penemuan biomarker dan personalisasi terapi kanker.

Biomarker didefinisikan sebagai karakteristik yang diukur sebagai indikator proses biologis normal, proses patogenik, atau respons farmakologis terhadap intervensi terapeutik (FDA-NIH Biomarker Working Group, 2016). Biomarker utama yang menjadi fokus klinis penapisan adenokarsinoma paru-paru meliputi alterasi genetik pada *Epidermal Growth Factor Receptor* (EGFR), *Anaplastic Lymphoma Kinase* (ALK), *B-RAF proto-oncogene, serine/threonine kinase* (BRAF), *ROS Proto-Oncogene 1* (ROS1), serta status profil imunoekspresi protein *Programmed Death-Ligand 1* (PD-L1) (Villalobos & Wistuba, 2017). Rentang prevalensi kemunculan penanda biologis ini bervariasi, di mana mutasi EGFR mencatatkan angka 10–15% pada pasien Barat dan melonjak hingga di atas 40% pada populasi Asia Timur, penataan ulang (*rearrangement*) fusi ALK berkisar antara 4–7%, mutasi titik BRAF sebesar 2–4%, alterasi fusi ROS1 sebanyak 1–2%, serta tingginya indeks positifitas ekspresi membran seluler dari ligand imunitas PD-L1 (Villalobos & Wistuba, 2017). Peran kelima biomarker tersebut sangat penting karena sebagian besar kasus kanker paru-paru terdeteksi pada tahap lanjut, sehingga pendeteksian dini dapat meningkatkan peluang kesembuhan pasien secara dramatis (Heryana et al., 2018). Dengan demikian, kelima biomarker tersebut menjadi instrumen target yang tepat untuk mempercepat penapisan komputasional dan meningkatkan peluang kesembuhan pasien kanker paru-paru adenokarsinoma melalui pendekatan diagnostik dan terapeutik yang lebih presisi.

Pada penelitian terdahulu yang diteliti oleh L. Das et al. (2023) menggunakan arsitektur *Artificial Neural Network* (ANN) dengan model *adaptive exponential link artificial neural network* untuk mendeteksi awal kanker payudara dengan membandingkan data nukleotida dari DNA yang terkait dengan kanker dan DNA yang

normal atau tidak terdapat kanker. Hasil dari penelitian tersebut menghasilkan akurasi dari metode klasifikasi sebesar 96,5% untuk 50 iterasi pelatihan dengan partisi pelatihan sebesar 65-35 *training-testing* yang divalidasi menggunakan 10 *fold cross-validation*. Namun, penelitian ini belum diarahkan pada kanker paru-paru adenokarsinoma dan menggunakan arsitektur ANN konvensional yang tidak secara khusus dirancang untuk menangani dependensi kontekstual jangka panjang dalam data sekuensial seperti DNA. Dalam konteks spesifik kanker paru-paru, urgensi penggunaan kecerdasan buatan pada level genetik telah dibuktikan oleh Almufareh et al. (2026), yang sukses mengimplementasikan model *Deep Learning* untuk mengklasifikasikan jenis *Lung Adenocarcinoma* secara murni dengan memanfaatkan profil modifikasi metilasi DNA dari tingkat genom.

Untuk menjawab kebutuhan komputasi yang mampu mengekstraksi sifat sekuensial dari DNA tersebut secara optimal, maka diusulkan implementasi *Bidirectional Long Short-Term Memory* (BiLSTM) dan *Bidirectional Gated Recurrent Unit* (BiGRU). Pemilihan arsitektur ini dilandasi oleh temuan komprehensif Wisesty et al. (2022), yang secara gamblang menyimpulkan bahwa BiLSTM dan BiGRU jauh lebih stabil dan akurat dibandingkan arsitektur *Convolutional Neural Network* (CNN) dalam mendeteksi tipe dan indeks mutasi pada sekuens DNA kanker paru-paru, sekaligus mencatatkan efisiensi waktu pemrosesan yang sangat tinggi. Ketangguhan struktur memori dua arah murni pada patologi yang sama ini divalidasi lebih lanjut oleh literatur terbaru dari Diao et al. (2025), di mana optimasi jaringan *bidirectional* terbukti memiliki kapabilitas yang sangat superior dalam mendeteksi dan mengklasifikasikan kanker paru-paru secara akurat. Selain terbukti andal pada patologi paru-paru, keunggulan model ini dalam menangani sekuens nukleotida juga sejalan dengan penelitian Zulhamsyah (2024), yang menunjukkan bahwa BiLSTM dan BiGRU beroperasi dengan sangat akurat dalam mendeteksi kanker payudara menggunakan data *biomarker* DNA, dengan akurasi *training*, *validation*, dan *testing* berturut-turut mencapai 100%, 99%, dan 98%. Rekam jejak keberhasilan ini diperkuat lebih lanjut melalui studi oleh Roslidar et al. (2026), yang menerapkan metode fusi keputusan (*decision fusion*) pada jaringan saraf dua arah (*bidirectional*) untuk klasifikasi kanker payudara berbasis sekuens DNA dan sukses meningkatkan performa akurasi sistem penapisan secara signifikan. Berpijak pada kokohnya landasan tersebut, peneliti berharap dapat memberikan kontribusi dalam pengembangan model prediksi kanker paru-paru adenokarsinoma menggunakan *biomarker* DNA. Penelitian ini diharapkan mampu menghasilkan instrumen komputasi berbasis bioinformatika yang tangguh dan berpresisi tinggi, sehingga dapat diandalkan sebagai sistem pendukung keputusan dalam proses penapisan mutasi genetik secara otomatis.

1.2 RUMUSAN MASALAH

Berdasarkan latar belakang yang telah diuraikan, permasalahan dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Perlunya merancang representasi data biomarker DNA ke dalam bentuk yang sekuensial yang kompatibel untuk diproses oleh model RNN.
2. Perlunya identifikasi dan validasi terhadap fitur-fitur biomarker DNA yang paling informatif dan memiliki nilai prediktif tinggi dalam membedakan kasus adenokarsinoma paru-paru, serta menganalisis kontribusinya terhadap peningkatan akurasi model RNN.
3. Diperlukan evaluasi komparatif di antara arsitektur BiLSTM dan BiGRU untuk mengetahui model mana yang lebih baik dalam akurasi dan kemampuan untuk menangkap dependensi jangka panjang pada data biomarker DNA untuk prediksi kanker paru-paru adenokarsinoma.

1.3 TUJUAN PENELITIAN

Adapun maksud dan tujuan dari penelitian ini adalah sebagai berikut:

1. Membangun model klasifikasi menggunakan data biomarker DNA.
2. Membandingkan performa BiLSTM dan BiGRU dalam menangani karakteristik sekuensial biomarker DNA.
3. Membuat pendeteksi kanker paru-paru adenokarsinoma dengan algoritma kecerdasan buatan.

1.4 MANFAAT PENELITIAN

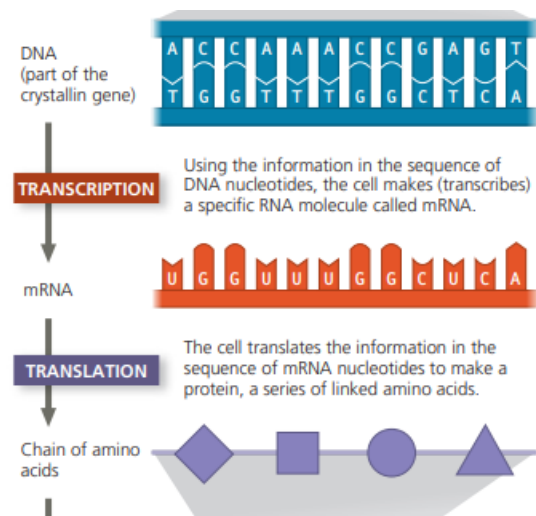
Manfaat yang diinginkan dari penelitian ini adalah memberikan evaluasi komparatif dan rekomendasi arsitektur *Deep Learning* yang optimal antara BiLSTM dan BiGRU dalam memproses data biomarker DNA untuk tugas prediksi kanker.

BAB II

TINJAUAN KEPUSTAKAAN

2.1 DOGMA SENTRAL BIOLOGI MOLEKULER

DNA merupakan molekul yang menyimpan informasi genetik dalam bentuk struktur heliks ganda, yang tersusun atas dua untai polinukleotida. Setiap nukleotida terdiri dari gula deoksiribosa, gugus fosfat, dan satu dari empat basa nitrogen: Adenin (A), Guanin (G), Sitosin (C), atau Timin (T) (Watson et al., 2014). Unit fungsional dalam DNA adalah gen, yaitu segmen tertentu yang mengandung instruksi untuk mensintesis protein. Proses pemanfaatan informasi ini, yang disebut ekspresi gen, terjadi melalui dua tahap utama: transkripsi dan translasi. Dalam proses ini, DNA tidak langsung digunakan sebagai cetakan untuk protein, melainkan ditranskripsikan terlebih dahulu menjadi RNA kurir (mRNA). Molekul RNA pada dasarnya berbeda dari DNA; RNA berupa untai tunggal, memiliki gula ribosa, dan menggunakan basa Urasil (U) sebagai pengganti Timin (T) (Alberts et al., 2015). Selanjutnya, sekuens mRNA diterjemahkan menjadi urutan asam amino yang menyusun suatu protein.



(Urry et al., 2022)

Gambar 2.1. Ilustrasi Proses Ekspresi Gen

1. Transkripsi adalah proses penyalinan informasi genetik dari DNA ke dalam bentuk RNA, menghasilkan mRNA yang akan digunakan dalam sintesis protein. Proses ini terjadi di dalam nukleus dan dikatalisis oleh enzim RNA polimerase (Urry et al., 2022).
2. Translasi adalah proses penerjemahan urutan nukleotida pada mRNA menjadi urutan asam amino yang membentuk protein. Proses ini berlangsung di

sitoplasma, dimulai saat mRNA berasosiasi dengan ribosom dan diakhiri saat ribosom mencapai kodon stop (Schlusser et al., 2024).

2.1.1 Biomarker

Biomarker merupakan karakteristik biologis yang diukur sebagai indikator proses normal, patogenik, atau respons terhadap paparan maupun terapi, dengan aplikasi penting seperti estimasi risiko, skrining, diagnosis, prognosis, prediksi respons terapi, dan pemantauan penyakit. Dalam onkologi, biomarker dapat berupa protein terkait kanker, mutasi gen, delesi, rearrangements, atau peningkatan salinan gen yang terdeteksi melalui assay berbasis darah atau memerlukan biopsi (Ou et al., 2021). Biomarker ideal bersifat biner atau kuantitatif, diukur dengan assay yang cepat, sensitif, spesifik, serta menggunakan spesimen yang mudah diperoleh. Perkembangan teknologi omik memungkinkan profil molekuler berskala besar yang mempercepat penemuan biomarker, meski masih terkendala biaya dan logistik pengumpulan sampel (Ng et al., 2023).

2.1.2 Biomarker Genomik pada Adenokarsinoma Paru-Paru

Dalam perspektif biologi molekuler, kelompok gen seperti *Epidermal Growth Factor Receptor* (EGFR), *Anaplastic Lymphoma Kinase* (ALK), *B-RAF proto-oncogene* (BRAF), *Programmed Death-Ligand 1* (PD-L1), dan *ROS proto-oncogene 1* (ROS1) merupakan elemen fungsional dasar yang terintegrasi secara universal di dalam genom manusia normal. Berdasarkan prinsip ekspresi gen, sekuens ini secara alami diekspresikan di berbagai jaringan tubuh yang sehat untuk mengodekan protein regulator pertumbuhan sel, jalur pensinyalan homeostasis, serta modulasi sistem imun (Alberts et al., 2015). Oleh karena itu, keberadaan atau ekspresi basal dari rangkaian gen fungsional ini dalam kondisi aslinya tidak dapat dikategorikan sebagai biomarker spesifik yang hanya mencirikan organ paru-paru yang berada dalam keadaan normal (Sung et al., 2023).

Status klinis dari sekuens nukleotida ini sebagai biomarker baru terbentuk ketika terjadi alterasi genetik atau mutasi somatik patologis (*driver mutations*) yang mengganggu fungsi regulasi fisiologis seluler. Perubahan struktural pada susunan basa nitrogen—baik berupa delesi ekson, substitusi titik, maupun fusi genetik—mentransformasikan sekuens fungsional tersebut menjadi onkogen aktif yang mendorong proliferasi sel tak terkendali pada kasus adenokarsinoma paru-paru (Dietz et al., 2022). Dengan demikian, pendekatan komputasi dalam penapisan biomarker pada penelitian ini tidak diarahkan untuk membedakan entitas nama atau rumpun gen itu sendiri, melainkan pada identifikasi pola mutasi spesifik di dalam sekuens nukleotida yang membedakan status onkogenik dari kondisi asli gen yang tidak mengalami mutasi

(Villalobos & Wistuba, 2017). Karakteristik fungsional serta manifestasi alterasi spesifik dari masing-masing gen target tersebut diuraikan secara rinci sebagai berikut:

1. EGFR

Reseptor faktor pertumbuhan epidermal (EGFR) adalah protein tirosin kinase dari keluarga ERBB yang terletak pada kromosom 7. Pada kondisi fisiologis normal, EGFR diaktifkan oleh ikatan ligan untuk memicu dimerisasi serta aktivasi jalur pensinyalan (seperti PI3K/AKT/mTOR) yang mengatur kelangsungan hidup sel. Pada kanker *Non-Small Cell Lung Cancer* (NSCLC), khususnya adenokarsinoma, sekuens gen EGFR sering mengalami mutasi yang menyebabkan reseptor aktif secara terus-menerus tanpa adanya ligan. Alterasi spesifik yang paling sering ditemukan sebagai biomarker patologis adalah delesi pada ekson 19 (*EGFR exon 19 deletion*) dan substitusi titik L858R pada ekson 21 (*EGFR L858R*). Mutasi ini memicu pertumbuhan sel ganas tak terkendali, namun pasien dengan karakteristik sekuens ini memiliki tingkat respons yang baik terhadap terapi target menggunakan *Tyrosine Kinase Inhibitor* (TKI) seperti gefitinib atau erlotinib (Villalobos & Wistuba, 2017).

2. ALK

Anaplastic Lymphoma Kinase (ALK) merupakan reseptor tirosin kinase transmembran yang secara normal berperan dalam perkembangan sistem saraf dan fungsi dasar regulasi seluler. Pada kanker paru-paru adenokarsinoma, biomarker patologis dari gen ini muncul akibat penyusunan ulang kromosom (*rearrangement*) yang menghasilkan mutasi fusi genetik. Fusi yang paling umum dijumpai adalah *EML4-ALK*, di mana gen ALK bergabung dengan gen *Echinoderm Microtubule-associated Protein-like 4* (EML4). Protein chimera EML4-ALK ini bersifat onkogenik dan memicu proliferasi sel ganas secara independen serta menghambat proses apoptosis. Fusi EML4-ALK umumnya lebih sering terjadi pada pasien berusia muda dan pasien bukan perokok (Villalobos & Wistuba, 2017).

3. BRAF

Onkogen BRAF yang terletak pada kromosom 7 mengkode kinase serin/treonin yang merupakan bagian vital dalam jalur pensinyalan sel normal RAS/RAF/MEK/ERK. Pada jaringan kanker, mutasi somatik pada sekuens gen ini mengakibatkan peningkatan fosforilasi secara abnormal sehingga mendorong proliferasi sel tumor. Karakteristik mutasi BRAF yang paling dominan dan menjadi target evaluasi klinis adalah mutasi *BRAFV600* (khususnya V600E),

di mana terjadi substitusi asam amino valin menjadi glutamat pada kodon 600. Alterasi mutasi ini lebih sering terjadi pada kasus adenokarsinoma paru-paru pada pasien dengan riwayat merokok. Meskipun memberikan prognosis yang lebih agresif, identifikasi perubahan sekuens ini penting untuk menentukan kelayakan pasien dalam menerima terapi target inhibitor spesifik (Villalobos & Wistuba, 2017).

4. PD-L1 / CD274

Programmed Death-Ligand 1 (PD-L1) adalah protein imun transmembran yang dikodekan oleh gen CD274. Pada kondisi homeostasis normal, PD-L1 berfungsi untuk menjaga toleransi sistem imun dengan cara berikatan pada reseptor PD-1 di sel T guna mencegah respons autoimun yang merusak jaringan sehat. Pada jaringan kanker, sel tumor memanfaatkan sistem ini melalui alterasi berupa amplifikasi atau overekspresi gen CD274 (*CD274 / PD-L1 related alterations*). Tingginya level ekspresi PD-L1 di permukaan sel kanker berfungsi sebagai mekanisme pertahanan mekanis, memungkinkan tumor menghindari serangan dari sistem kekebalan tubuh pasien. Penilaian alterasi terkait sekuens CD274 (PD-L1) merupakan indikator wajib untuk menentukan efektivitas penerapan pengobatan berbasis imunoterapi (Villalobos & Wistuba, 2017).

5. ROS1

ROS1 adalah reseptor tirosin kinase dari keluarga reseptor insulin yang terletak pada kromosom 6, yang dalam kondisi normal memiliki peran memfasilitasi diferensiasi sel epitel. Sebagai biomarker kanker, sekuens gen ROS1 dievaluasi jika mengalami *rearrangement* atau mutasi fusi genetik dengan partner lain, seperti CD74 atau SLC34A2. Mirip dengan mekanisme ALK, perombakan struktur genetik ini menciptakan protein chimera onkogenik pendorong kanker. Mutasi atau fusi ROS1 ini umumnya terjadi pada persentase kecil pasien adenokarsinoma dan secara tipikal ditemukan pada pasien perempuan, berusia muda, bukan perokok, serta dievaluasi secara eksklusif jika tidak ditemukan mutasi pada gen EGFR dan ALK (Villalobos & Wistuba, 2017).

2.2 BAHASA PEMROGRAMAN DAN PUSTAKA PENDUKUNG

2.2.1 Python

Python adalah bahasa pemrograman *open-source* dan *cross-platform* yang pertama kali dirilis pada 1991 dan kini berkembang pesat hingga versi 3.7.0, dengan CPython sebagai implementasi referensi sekaligus default yang paling luas digunakan. Sebagai bahasa *multi-purpose*, Python mendukung berbagai bidang seperti komputasi

ilmiah, simulasi, dan pengembangan web melalui *framework* seperti Django (Mehare et al., 2023). Popularitasnya semakin kokoh di era *machine learning*, di mana perusahaan besar memanfaatkannya untuk segmentasi konsumen, otomatisasi proses, dan pengembangan aplikasi. Keunggulan Python terletak pada ekosistem pustaka dan *framework* yang terus berkembang, sintaksis sederhana, serta waktu pengembangan yang singkat, menjadikannya sangat ideal untuk aplikasi *data science* dan *machine learning* (Halvorsen, 2019).

2.2.2 BioPython

Biopython adalah pustaka sumber terbuka Python untuk bioinformatika yang didirikan pada 1999 oleh Open Bioinformatics Foundation. Pustaka ini menyediakan modul untuk analisis sekuens biologis, akses basis data seperti *National Center for Biotechnology Information* (NCBI), penanganan format standar (FASTA, GenBank, *Basic Local Alignment Search Tool* (BLAST)), serta algoritma *alignment* dan pemodelan struktur protein. Pengembangannya terus berlanjut, salah satunya melalui pustaka tambahan FoundSeq v1.0 yang mengintegrasikan layanan eksternal seperti ExPASy, BLAST, UniProt, dan DrugBank melalui API terpadu, sehingga mempermudah analisis dan memperluas aplikasi dalam genomik, drug discovery, dan penelitian translasional (Sá & Nogueira, 2024).

2.3 ALGORITMA DAN PEMODELAN PADA DATA SEKUENSIAL

2.3.1 BLAST dari NCBI

BLAST adalah algoritma heuristik untuk pencarian kemiripan sekuens biologis dengan membandingkan sekuens kueri terhadap database referensi seperti NCBI nr/nt melalui *local alignment*. Meskipun bersifat heuristik dan memiliki potensi ketidakpastian, BLAST tetap menjadi standar dalam bioinformatika karena efisiensi dan keandalannya. Dalam biosekuriti, BLAST digunakan untuk menilai kedekatan sekuens dengan patogen atau toksin yang diatur, di mana hasil *alignment* menjadi dasar klasifikasi risiko. Keterbatasan seperti ambiguitas taksonomi pada database NCBI dapat memengaruhi akurasi, sehingga interpretasi akhir sering memerlukan validasi ahli, terutama untuk sekuens rekayasa genetika atau chimerism alami.

2.3.2 MAFFT

MAFFT (*Multiple Alignment using Fast Fourier Transform*) adalah salah satu alat bioinformatika paling banyak digunakan untuk *multiple sequence alignment* (MSA). Berdasarkan Katoh et al. (2019), MAFFT memanfaatkan algoritma Fast Fourier Transform (FFT) untuk mendeteksi kemiripan sekuens secara efisien, dengan dukungan berbagai strategi seperti *progressive alignment* (FFT-NS-2), *iterative refinement*

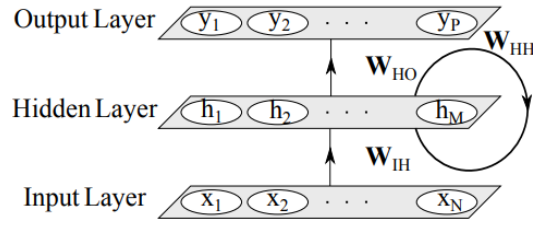
(G-INS-i), dan *consistency-based approaches* (L-INS-i). Layanan daringnya tidak hanya menawarkan algoritma penyelarasan yang canggih, tetapi juga antarmuka interaktif yang memungkinkan pengguna memilih serta memvisualisasikan sekuens secara dinamis. Keunggulan MAFFT terletak pada kemampuannya menangani data berskala besar dengan panjang sekuens variatif maupun komposisi basa tidak seimbang, sambil tetap menjaga akurasi hasil *alignment*. Kombinasi efisiensi komputasi dan fungsionalitas interaktif menjadikan MAFFT *indispensable* dalam studi filogenetik, analisis domain fungsional, maupun penelitian evolusi molekuler, sekaligus mendukung kebutuhan peneliti dalam menghadapi *big data* biologis (Katoh et al., 2019).

2.3.3 Deep Learning

Deep Learning adalah sub-bidang dari *machine learning* yang menunjukkan performa unggul terutama pada pemrosesan data tidak terstruktur seperti citra, teks, dan audio. Berbeda dengan metode konvensional, pendekatan ini mampu mempelajari representasi fitur bertingkat (*hierarchical feature representation*) secara otomatis dari data mentah, sehingga menghasilkan akurasi yang lebih tinggi dibanding teknik tradisional. Kemajuan *deep learning* didorong oleh ketersediaan *big data* dan peningkatan kapasitas komputasi perangkat keras, khususnya pemanfaatan Graphics Processing Units (GPU) dan Tensor Processing Units (TPU) yang mempercepat proses pelatihan model secara signifikan (Mathew et al., 2021). Dalam bidang bioinformatika, *deep learning* telah diterapkan untuk prediksi struktur protein, deteksi penyakit genetik, dan analisis biomarker, termasuk dalam penelitian ini untuk mengklasifikasikan jenis kanker berdasarkan pola biomarker.

2.3.4 RNN

Recurrent Neural Networks (RNN) merupakan kelas jaringan saraf tiruan yang dirancang khusus untuk memproses data sekuensial. Berbeda dengan jaringan *feedforward* tradisional, RNN memiliki kemampuan mempertahankan *state* internal yang berisi informasi tentang elemen-elemen sebelumnya dalam sekuens. Karakteristik ini membuat RNN sangat cocok untuk tugas-tugas seperti pemrosesan bahasa alami, pengenalan ucapan, dan analisis deret waktu. (Merrill et al., 2020).



(Salehinejad et al., 2017)

Gambar 2.2. Arsitektur RNN

Gambar tersebut menunjukkan arsitektur dasar Recurrent Neural Network (RNN) seperti dijelaskan oleh (Salehinejad et al., 2017), yang terdiri dari lapisan input, tersembunyi, dan output. Neuron input $x_1, x_2, \dots, x_N$ terhubung ke lapisan tersembunyi $h_1, h_2, \dots, h_M$ melalui bobot W_{IH} , kemudian diteruskan ke output $y_1, y_2, \dots, y_P$ melalui W_{HO} . Keunikan RNN terletak pada umpan balik di lapisan tersembunyi melalui bobot rekursif W_{HH} yang memungkinkan jaringan menyimpan informasi dari langkah waktu sebelumnya. Mekanisme ini membuat RNN mampu menangkap dependensi temporal pada data sekuensial seperti teks, suara, atau deret waktu, meski rentan terhadap masalah *vanishing gradient* yang kemudian mendorong pengembangan LSTM dan GRU.

$$h_t = f_H(o_t) \quad (2.1)$$

$$o_t = W_{IH}x_t + W_{HH}h_{t-1} + b_h \quad (2.2)$$

$$y_t = f_O(W_{HO}h_t + b_o) \quad (2.3)$$

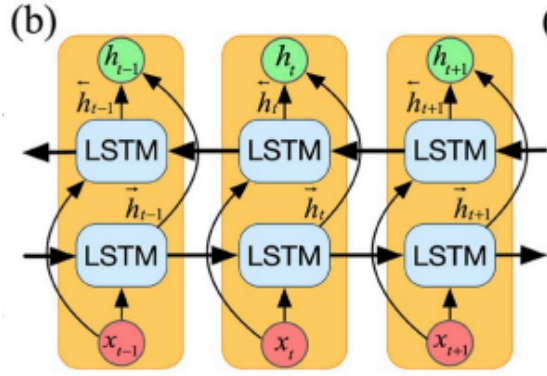
Berdasarkan persamaan tersebut setiap keterangan simbol yang ada akan dijelaskan sebagai berikut :

- h_t adalah unit *hidden* terhadap waktu t .
- f_H adalah *activation function* pada lapisan *hidden*.
- o_t adalah prediksi keluaran terhadap waktu t berdasarkan vektor masukan.
- W_{IH} adalah matriks bobot yang menjadi penghubung lapisan masukan ke lapisan *hidden*.
- x_t adalah vektor masukan terhadap waktu t .
- W_{HH} adalah matriks bobot yang terhubung ke satu sama lain melalui waktu dengan koneksi *recurrent*.

- b_h adalah vektor bias dari unit *hidden*.
 - y_t adalah lapisan keluaran yang sebenarnya.
 - f_O adalah *activation function* pada lapisan keluaran.
 - W_{HO} adalah matriks bobot yang menghubungkan unit *hidden* ke lapisan keluaran.
 - b_o adalah vektor bias dari lapisan keluaran.
1. Jaringan Saraf Berulang Penuh (Fully Recurrent Neural Network/FRNN), dikembangkan pada 1980-an dengan dua lapisan utama: masukan dan keluaran, di mana setiap unit input terhubung penuh ke seluruh unit output. Mekanisme umpan baliknya memungkinkan penyimpanan informasi dari langkah sebelumnya.
 2. Memori Jangka Panjang dan Pendek *Long Short-Term Memory* (LSTM), pengembangan RNN untuk mengatasi *vanishing gradient* dengan gerbang lupa yang mengatur penyimpanan atau penghapusan informasi. Varian BiLSTM memproses data maju dan mundur untuk pemahaman konteks yang lebih baik.
 3. Unit Berulang Tergerbang (Gated Recurrent Units/GRU), versi sederhana LSTM tanpa gerbang keluaran dan dengan lebih sedikit parameter sehingga lebih efisien, namun tetap memiliki kinerja setara dalam tugas seperti pengenalan suara dan teks. Tersedia juga varian *bidirectional*.
 4. Jaringan Saraf Berulang Dua Arah (*Bidirectional Recurrent Neural Network/BRNN*), memanfaatkan konteks masa lalu dan masa depan dengan memproses urutan data dari dua arah, dan bila digabung dengan LSTM mampu memodelkan ketergantungan jangka panjang lebih efektif.

2.3.5 BiLSTM

Bidirectional Long Short-Term Memory (BiLSTM) adalah varian *RNN* yang memproses data maju dan mundur untuk menangkap konteks masa lalu dan masa depan sekaligus, sehingga efektif mempelajari ketergantungan jangka panjang (Bian et al., 2020). Dalam analisis biomarker, BiLSTM membantu mengenali pola kompleks pada sekuens gen atau protein.



(Bian et al., 2020)

Gambar 2.3. Arsitektur BiLSTM

BiLSTM merupakan pengembangan LSTM yang memproses data maju dan mundur sehingga setiap langkah waktu dipengaruhi konteks masa lalu dan masa depan. Pendekatan dua arah ini menghasilkan representasi lebih komprehensif dan efektif untuk mempelajari ketergantungan jangka panjang, sehingga meningkatkan akurasi analisis deret waktu seperti pemodelan status baterai lithium-ion (Bian et al., 2020).

Persamaan gerbang LSTM didefinisikan sebagai berikut:

Gerbang Input (i_t):

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (2.4)$$

Gerbang Forget (f_t):

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (2.5)$$

Gerbang Output (o_t):

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (2.6)$$

Kandidat Memori Sel ($\tilde{c}_t$):

$$\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (2.7)$$

Update Memori Sel (C_t):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{c}_t \quad (2.8)$$

Update Hidden State (h_t):

$$h_t = o_t \odot \tanh(C_t) \quad (2.9)$$

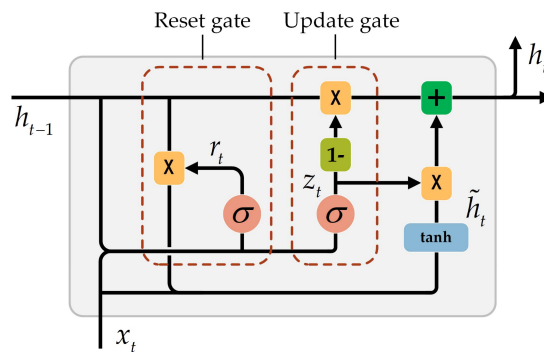
Keterangan:

- σ adalah fungsi aktivasi sigmoid,

- $\odot$ menandakan perkalian elemen-bijaksana (*element-wise product*),
- $\mathbf{W}$ adalah matriks bobot,
- $\mathbf{b}$ adalah vektor bias.

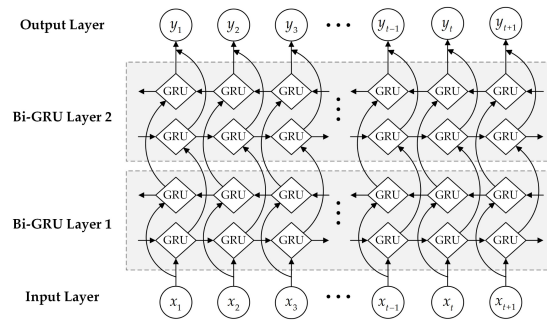
2.3.6 BiGRU

Gated Recurrent Unit (GRU) merupakan varian dari *Recurrent Neural Network* (RNN) yang dirancang untuk mengatasi keterbatasan RNN konvensional, khususnya dalam menangani masalah *vanishing gradient* dan *exploding gradient* serta ketidakmampuan dalam memodelkan dependensi jangka panjang (*long-range dependencies*). Diusulkan oleh *Chao et al.* [33] sebagai penyederhanaan dari arsitektur LSTM, GRU mengkonsolidasi gerbang lupa (*forget gate*) dan gerbang masukan (*input gate*) pada LSTM menjadi sebuah gerbang tunggal yang disebut *update gate*, serta menggabungkan keadaan sel (*cell state*) dan keadaan tersembunyi (*hidden state*). Struktur sirkular pada GRU memungkinkan keluaran pada setiap langkah waktu t ditentukan oleh masukan saat ini $\mathbf{x}_t$ dan keadaan tersembunyi sebelumnya $\mathbf{h}_{t-1}$, sehingga menjadikannya cocok untuk tugas-tugas yang melibatkan ketergantungan temporal seperti pemodelan urutan, pelabelan berurutan (*sequence labeling*), dan segmentasi kata.



(Li et al., 2020)

Gambar 2.4. Arsitektur GRU



(Li et al., 2020)

Gambar 2.5. Arsitektur BiGRU

Bidirectional Gated Recurrent Unit (Bi-GRU) merupakan pengembangan GRU yang memproses data sekuensial secara maju ($t = 1 \rightarrow T$) dan mundur ($t = T \rightarrow 1$) untuk menangkap konteks masa lalu dan masa depan secara bersamaan. Pada setiap langkah t , keluaran kedua arah digabungkan melalui *concatenation* menjadi *hidden state* $h_t = [\vec{h}_t; \overleftarrow{h}_t]$, sementara mekanisme *reset gate* dan *update gate* memungkinkan pemeliharaan informasi penting serta mengurangi masalah *vanishing gradient*, menjadikannya efisien untuk analisis deret waktu, pemrosesan bahasa alami, dan deteksi kanker dalam penelitian ini.

2.3.7 TensorFlow

TensorFlow adalah *framework open-source* untuk mendefinisikan dan mengeksekusi algoritma pembelajaran mesin secara efisien pada berbagai platform, dari perangkat seluler hingga sistem terdistribusi berskala besar dengan dukungan GPU atau TPU (Abadi et al., 2015). Dalam bioinformatika, TensorFlow digunakan untuk membangun model deep learning, seperti RNN dengan lapisan LSTM atau GRU, guna menganalisis data biomarker DNA pada deteksi kanker paru-paru adenokarsinoma. Data sekuensial direpresentasikan sebagai tensor dan diproses untuk mengklasifikasikan sampel kanker dan non-kanker, dengan proses pelatihan yang dioptimalkan melalui komputasi gradien otomatis dan akselerasi perangkat keras.

2.4 PENELITIAN TERKAIT

Tabel 2.1. Penelitian Terkait

No	Penelitian	Dataset	Hasil
1.	Wisesty dkk. <i>Join Classifier of Type and Index Mutation on Lung Cancer DNA Using Sequential Labeling Model</i>	Dataset yang digunakan adalah sekuens DNA mutasi kanker paru-paru untuk klasifikasi tipe dan indeks mutasi.	Hasil penelitian ini menunjukkan performa menggunakan Sequential Labeling Model dengan tingkat akurasi mencapai 94,20% .
2.	Zulhamsyah Deteksi Kanker Payudara Menggunakan Data Biomarker DNA dengan Pendekatan <i>Recurrent Neural Network</i>	Dataset yang digunakan adalah data biomarker DNA untuk klasifikasi pada kasus kanker payudara .	Penelitian ini menggunakan arsitektur RNN berupa BiLSTM dan BiGRU yang memperoleh akurasi sebesar 95,21% .
3.	Almufareh dkk. <i>Deep-Learning-Based Classification of Lung Adenocarcinoma and Squamous Cell Carcinoma Using DNA Methylation Profiles</i>	Dataset yang digunakan adalah profil metilasi DNA yang terbagi dalam 2 kelas yaitu <i>Lung Adenocarcinoma</i> dan <i>Squamous Cell Carcinoma</i> .	Hasil dari penelitian ini mengusulkan klasifikasi berbasis Deep Learning dengan tingkat akurasi mencapai 96,85% .
4.	Diao dkk. (2025) <i>Optimizing Bi-LSTM networks for improved lung cancer detection accuracy</i>	Dataset medis dan ekstraksi fitur biomarker untuk deteksi dan klasifikasi kanker paru-paru.	Hasil dari penelitian ini membuktikan bahwa arsitektur <i>Deep Learning</i> berbasis BiLSTM murni memiliki performa yang sangat superior dalam mendeteksi kanker paru-paru, dengan tingkat akurasi mencapai 99,89% .

No	Penelitian	Dataset	Hasil
5.	Roslidar dkk. <i>Fusion of bidirectional neural networks decision for a high accuracy breast cancer detection based on DNA biomarker</i>	Dataset yang digunakan adalah data biomarker DNA untuk deteksi pada kasus kanker payudara .	Hasil dari penelitian ini membuktikan bahwa metode <i>Fusion of Bidirectional Neural Networks</i> memiliki performa tinggi dengan akurasi 98,20 % .

BAB III METODOLOGI PENELITIAN

3.1 WAKTU DAN LOKASI PENELITIAN

Penelitian ini akan bertempat di Ruang Lab Sistem Informasi dan Database. Waktu yang dibutuhkan agar penelitian ini dapat diimplementasikan adalah 5 bulan terhitung dari bulan Agustus 2025 hingga Desember 2025.

3.2 JADWAL PELAKSANAAN

Untuk lebih memahami alur waktu dari penelitian ini, penulis telah menyusun sebuah jadwal yang rinci yang bisa dilihat pada Tabel 3.1.

Tabel 3.1. Jadwal pelaksanaan penelitian

No	Kegiatan	Bulan Ke -															
		Agustus				September				Oktober				November			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Studi literatur																
2	Pengumpulan data																
3	Pemrosesan data																
4	Membangun model																
5	Melatih model																
6	Analisis hasil																
7	Penyusunan laporan akhir																

3.3 ALAT DAN BAHAN

Alat dan bahan yang digunakan dalam penelitian ini terdiri dari beberapa perangkat keras (*hardware*) dan perangkat lunak (*software*). Data yang digunakan dalam penelitian ini merupakan *dataset* biomarker DNA paru-paru normal dan kanker paru-paru adenokarsinoma. Jenis gen yang menjadi biomarker untuk paru-paru normal dan kanker paru-paru adenokarsinoma disesuaikan dengan protein yang dihasilkan dan pengaruhnya terhadap kesehatan paru-paru.

3.3.1 Sumber Dataset

Sumber dataset biomarker sekuensial nukleotida untuk paru-paru normal dan kanker adenokarsinoma paru-paru yang digunakan dalam penelitian ini berasal dari *National Center for Biotechnology Information* (NCBI), secara spesifik menggunakan repositori *Nucleotide* (GenBank). NCBI merupakan basis data genomik komprehensif berskala global di bawah National Institutes of Health yang berfungsi sebagai repositori utama untuk data sekuens nukleotida biomolekul, seperti untaian DNA dan RNA untuk

penelitian berkaitan dengan bidang biologi, kesehatan, dan bioteknologi (Sayers et al., 2024).

Pemilihan NCBI *Nucleotide* didasarkan pada ketersediaan data sekuens linear satu dimensi (1D) dalam format FASTA yang merefleksikan urutan alfabetik basa nukleotida (*Adenine*, *Cytosine*, *Guanine*, *Thymine*) secara presisi, sehingga siap ditransformasikan ke dalam representasi numerik untuk model *deep learning*. Secara spesifik, penelitian ini mengekstraksi data sekuens murni spesies *Homo sapiens* dari lima gen biomarker utama kanker paru-paru adenokarsinoma, yaitu *Epidermal Growth Factor Receptor* (EGFR), *Anaplastic Lymphoma Kinase* (ALK), *B-RAF proto-oncogene, serine/threonine kinase* (BRAF), *Programmed Death-Ligand 1* (PD-L1), dan *ROS Proto-Oncogene 1* (ROS1) guna menjaga validitas pengujian model *Bidirectional Long Short Term Memory* (BiLSTM) dan *Bidirectional Gated Recurrent Unit* (BiGRU).

3.3.2 Perangkat Keras

Lenovo ideapad slim 3 dengan Intel® Core™ i3-10110U CPU @ 2.10 GHz, RAM 8GB, *Solid State Drive* (SSD) 256GB, Fedora Linux 41 (KDE Plasma) , Mesa Intel® UHD Graphics

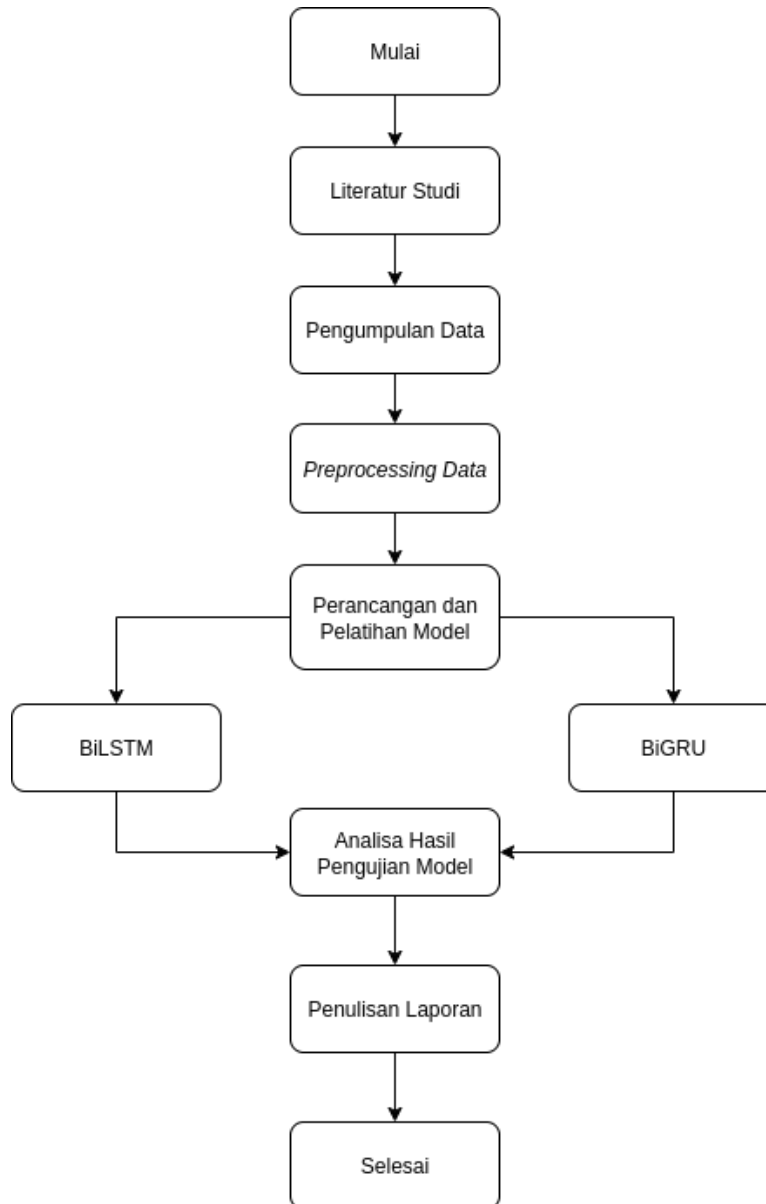
3.3.3 Perangkat Lunak

- a. Fedora Linux 41 (KDE Plasma)
- b. Visual Studio Code versi 1.98.0
- c. Python versi 3.13.2
- d. TensorFlow versi 2.20.0rc0
- e. BioPython 1.85
- f. NumPy 2.3.2
- g. Google Colaboratory
- h. Matplotlib versi 3.10.5
- i. scikit-learn versi 1.7.1

3.4 METODE PENELITIAN

Penelitian ini akan mengikuti beberapa tahapan yang dirancang secara sistematis untuk mencapai tujuan yang telah ditetapkan. Setiap tahap akan dilaksanakan dengan seksama untuk memastikan hasil yang akurat dan bermanfaat. Setiap tahapan dari alur

tahapan penelitian yang diusulkan dapat dilihat secara rinci pada Gambar 3.1. Pada Gambar 3.1 terdapat beberapa tahapan yang akan dilakukan dalam penelitian ini, yaitu studi literatur, pengumpulan data, prapemrosesan data, perancangan model, pelatihan model, analisis hasil pengujian model, dan penulisan laporan.



Gambar 3.1. Diagram Alir Penelitian

3.4.1 Studi Literatur

Studi literatur dilakukan dengan cara mempelajari landasan teori dari berbagai referensi jurnal, buku, dan sumber akademik dari penelitian terdahulu yang terkait dengan arsitektur *recurrent neural network* dengan metode *bidirectional gated recurrent unit* dan *bidirectional long short term memory* untuk meneliti dataset sekuensial

layaknya data biomarker DNA. Dengan dilakukannya studi literatur, peneliti dapat memperoleh wawasan yang lebih luas untuk menyusun kerangka penelitian serta memperkuat landasan teori yang mendukung penelitian ini.

3.4.2 Pengumpulan Data

Pada tahap ini, pengumpulan dataset dilakukan dengan menghimpun data dari repositori NCBI dalam bentuk berkas berformat FASTA. Tahap ekstraksi awal difokuskan secara spesifik pada sekuens DNA dari lima biomarker utama indikator kanker paru-paru adenokarsinoma, yaitu EGFR, ALK, BRAF, PD-L1, dan ROS1.

Setelah himpunan data awal diperoleh, tahapan verifikasi karakteristik sekuens dieksekusi memanfaatkan algoritma pencarian BLAST (*Basic Local Alignment Search Tool*) yang disediakan oleh NCBI. Penggunaan BLAST berfungsi sebagai instrumen uji silang untuk mengonfirmasi keaslian homogenitas biologis target sekuens. Selanjutnya, data melalui pemilahan dengan tiga tahapan penyaringan, yaitu:

1. **Penyaringan Spesies:** Isolasi data dilakukan secara eksklusif untuk memastikan seluruh sekuens murni berasal dari taksonomi *Homo sapiens*, mengeliminasi varian homolog dari organisme lain.
2. **Penyaringan Panjang Sekuens:** Menjaga kestabilan alokasi memori perangkat keras saat pelatihan dengan membatasi dan tidak menyertakan sekuens dengan dimensi puluhan ribu pasang basa yang ekstrem.
3. **Penyaringan Gen:** Melakukan verifikasi silang tipe gen beserta bentuk mutasinya untuk meminimalisasi risiko galat penempatan data (*misplaced data*) yang berpotensi mendistorsi metrik evaluasi.

Visualisasi dari struktur data mentah hasil akhir tahapan pengumpulan ini dapat dilihat pada Gambar 3.2.

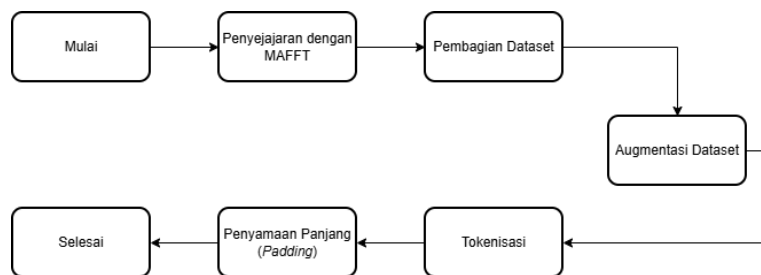


Gambar 3.2. Struktur Data Mentah

Berdasarkan Gambar 3.2, berkas berformat FASTA yang lolos seleksi memuat dua atribut utama: *header* (kode akses unik Gen ID beserta deskripsi nama lengkap gen) dan sekuens (runtutan basa nukleotida alfabetik murni A, C, G, T) yang akan dialirkan menuju fase prapemrosesan.

3.4.3 Prapemrosesan Data

Data nukleotida murni hasil pengumpulan masih memiliki variasi spasial yang tidak seimbang. Oleh karena itu, dijalankan serangkaian tahapan prapemrosesan seperti dipetakan pada Gambar 3.3 untuk mentransformasikan sekuens alfabetik tersebut ke dalam representasi bentuk tensor numerik yang siap untuk dilatih.



Gambar 3.3. Alur Prapemrosesan

3.4.3.1 Penyejajaran dengan MAFFT

Langkah awal dilakukan dengan menyelaraskan struktur urutan sekuens menggunakan perangkat lunak MAFFT. Algoritma ini memindai kemiripan sekuens

secara matematis dan menggeser posisi basa nukleotida sedemikian rupa guna mencapai titik kesejajaran struktural maksimal. Penyejajaran berganda ini menghasilkan matriks sekuens di mana residu homolog (termasuk titik mutasi tunggal, insersi, maupun delesi) berada pada koordinat kolom yang ekuivalen antara kelas normal dan kanker, memberikan fondasi pola konsisten bagi ekstraksi fitur model.

3.4.3.2 Pembagian Dataset

Untuk menjamin pengujian performa kerja model yang objektif, pembagian dataset menghindari pengacakan baris konvensional (*random train-test split*) yang rentan memicu fenomena kebocoran data (*data leakage*). Risikonya, fragmen sekuens dari klon sumber file yang sama dapat tersebar simultan ke dalam set latih dan uji, sehingga menciptakan metrik akurasi semu (*overoptimistic metrics*). Sebagai solusinya, riset ini menerapkan teknik *Stratified Source Split* berbasis grup sumber berkas dengan pembagian rasio proporsional: 80% sebagai Data Latih, 10% sebagai Data Validasi, dan 10% sebagai Data Uji. Teknik stratifikasi ini mengisolasi variasi sekuens dari satu sumber file agar tidak menyeberang antar subset pengujian.

3.4.3.3 Augmentasi Data

Keterbatasan ketersediaan sampel sekuens mutasi mentah pada data klinis publik memicu ketidakseimbangan data (*data imbalance*) yang dapat menimbulkan bias mayoritas pada model. Riset ini mengusulkan dua teknik augmentasi berbasis kaidah genetika molekuler untuk memperluas variansi data tanpa merusak integritas biologisnya:

1. **Mutasi Sinonim (*Synonymous Codon Substitution*):** Substitusi huruf basa nukleotida pada triplet kodon dengan kodon sinonimnya berdasarkan tabel kode genetik standar. Operasi ini memodifikasi arsitektur susunan huruf (genotipe) tanpa mengubah struktur ekspresi asam amino (fenotipe).
2. **Insersi / Delesi Kecil (*Small Indels*):** Menginjeksikan peluang probabilitas acak sebesar 2% untuk menyisipkan atau menghapus satu hingga tiga pasang basa guna mensimulasikan kegagalan pergeseran bingkai membaca (*frameshift errors*) atau variasi *noise* alat sekuensing.

3.4.3.4 Tokenisasi

Karakter alfabetik string (A, C, G, T) tidak diolah langsung oleh komputasi graf jaringan saraf. Sekuens ditransformasikan menjadi urutan integer numerik diskrit menggunakan objek *Tokenizer*. Objek pembacaan beserta pemetaan kelas

(*label mapping*) ini disimpan secara permanen ke dalam format *.pickle* terpisah pada masing-masing skenario volume augmentasi. Langkah ini berguna untuk mengantisipasi pergeseran kosakata (*vocabulary shift*), karena jika inisialisasi ulang indeks angka terjadi saat pengujian data eksternal di masa mendatang, model akan salah menginterpretasikan sekuens sebagai deret angka acak tanpa makna fungsional fisis.

3.4.3.5 Penyamaan Panjang (*Padding*)

Untuk menyatukan dimensi spasial matriks tensor agar seragam, diaplikasikan proses *padding*. Ambang batas *Max Sequence Length* (MSL) dihitung dinamis mengambil nilai persentil ke-99 dari total panjang data latih. Demi menjaga stabilitas alokasi memori grafis perangkat keras, batas atas MSL dikunci secara ketat *hardware cap* pada dimensi 3000 pasang basa. Sekuens di bawah batas tersebut akan diberikan nilai nol pada struktur akhir untaian sekuens (*post-padding*).

3.4.4 Perancangan dan Pelatihan Model

Tahapan komputasi utama dieksekusi dengan merancang, membangun, dan melatih arsitektur *recurrent neural network* (RNN) dari nol. Model yang diajukan secara spesifik difokuskan pada kapabilitas pemahaman sekuensial temporal jarak jauh dari jaringan saraf berulang dua arah (*Bidirectional RNN*).

Penyusunan arsitektur *deep learning* ini dibangun secara sekuensial melewati beberapa lapisan fungsional berikut:

1. **Lapisan *Embedding*:** Memetakan indeks integer diskrit masukan dari *tokenizer* ke dalam representasi vektor ruang berdimensi 16, membantu model mempelajari kedekatan semantik hubungan antar-basa nukleotida.
2. **Lapisan *Bidirectional*:** Vektor representasi hasil *embedding* langsung dialirkan menuju lapisan memori dua arah berkapasitas 128 unit. Pendekatan dua arah dipilih selaras dengan sifat komplementer untaian ganda DNA yang dapat dibaca secara simultan (arah 5' ke 3' dan sebaliknya) guna menangkap dependensi kontekstual jangka panjang lapisan memori secara terpisah, yaitu menggunakan metode BiLSTM pada arsitektur pertama dan BiGRU pada arsitektur kedua.
3. **Lapisan Klasifikasi Akhir (*Dense*):** Hasil dari *recurrent layer* diratakan melalui operasi *Global Max Pooling 1D* untuk mengambil fitur representasi terkuat, lalu diteruskan ke lapisan *Dense* perantara berkapasitas 32 neuron, sebelum bermuara pada satu neuron *output* tunggal beraktivasi *Sigmoid* untuk menghasilkan probabilitas biner klasifikasi kelas Normal atau Kanker.

Konfigurasi kompilasi model dioptimalkan menggunakan fungsi *loss Binary Focal Crossentropy* terspesialisasi digabungkan dengan pemberian bobot kelas (*Class Weights*) untuk memaksa penalti algoritma berfokus pada kesalahan deteksi kelas kanker yang berjumlah minoritas. Pada optimasi perangkat keras, peniadaan parameter *recurrent dropout* dilakukan guna mengaktifkan fungsionalitas pustaka akselerasi kernel NVIDIA CuDNN secara penuh untuk mempersingkat waktu pelatihan. Proses pelatihan dikontrol ketat oleh mekanisme regulasi *Early Stopping* yang akan menghentikan iterasi apabila metrik kerugian validasi tidak menunjukkan perbaikan konvergensi selama 10 langkah berturut-turut (*patience=10*).

3.4.5 Analisis Hasil Pengujian Model

Setelah proses pengujian selesai, dilakukan evaluasi dan perbandingan hasil dari kedua model yang telah dibangun dengan menggunakan metrik evaluasi berupa akurasi, *loss*, *precision*, *recall*, dan *F1-score*. Seluruh metrik tersebut diuji menggunakan subset data uji murni eksternal yang belum pernah dikenali oleh model sebelumnya. Selain itu, visualisasi kurva pembelajaran (*learning curve*) untuk metrik *loss* dan akurasi sepanjang iterasi akan dianalisis secara mendalam guna mendeteksi ada tidaknya indikasi masalah kegagalan generalisasi seperti gejala *overfitting* atau *underfitting*.

3.4.6 Penulisan Skripsi

Pada tahap ini, penulis akan menuliskan skripsi yang berisi semua fase penelitian yang telah dilakukan yang diawali dengan studi literatur, pengumpulan dataset, prapemrosesan dataset, penjabaran model BiLSTM dan BiGRU serta analisis hasil dari pelatihan model yang setelah itu dilakukan pengujian yang menjadikan dasar perbandingan performa kedua model yang digunakan. Kemudian dituliskan ke dalam hasil penelitian yang didapatkan menggunakan model BiLSTM dan BiGRU berupa laporan pelatihan dan pengujian model.

BAB IV

HASIL DAN PEMBAHASAN

Hasil dari penelitian ini adalah membangun model *deep learning* dengan pendekatan *Recurrent Neural Network* (RNN) menggunakan metode *Bidirectional Long Short Term Memory* (BiLSTM) dan *Bidirectional Gated Recurrent Unit* (BiGRU) yang memiliki kemampuan dalam melakukan deteksi kanker paru-paru adenokarsinoma menggunakan data biomarker DNA yang terbagi ke dalam dua kelas biner, yaitu kelas sekuens kontrol (tanpa mutasi) dan kelas sekuens mutasi patologis (onkogenik). Representasi kedua kelas ini diekstraksi dari rumpun gen target yang sama, yaitu *Epidermal Growth Factor Receptor* (EGFR), *Anaplastic Lymphoma Kinase* (ALK), *B-RAF Proto-Oncogene, Serine/Threonine Kinase* (BRAF), *Programmed Death-Ligand 1* (PD-L1), dan *ROS Proto-Oncogene 1* (ROS1), untuk memastikan model mengidentifikasi pola alterasi urutan nukleotida penyusun kanker, bukan sekadar membedakan variasi entitas antar-gen yang berbeda. Performa kedua metode akan dievaluasi dengan cara membandingkan tingkat akurasi dan *loss*.

4.1 EVALUASI MODEL

Evaluasi model dilakukan untuk mengukur performa dari masing-masing arsitektur, yaitu BiLSTM dan BiGRU, dalam melakukan klasifikasi sekuens antara data mutasi onkogenik dan data kontrol tanpa mutasi. Evaluasi bertujuan untuk menilai sejauh mana model dapat menggeneralisasi pola yang telah dipelajari dari data latih ke data uji. Tahap ini dilakukan dengan cara menguji daya generalisasi model yang telah dilatih menggunakan data uji murni (*Testing Set*) yang diisolasi sejak awal. Untuk mengukur secara objektif kemampuan model dalam mengenali pola dari dataset secara baik, pemaparan hasil evaluasi dibagi menjadi dua penyajian utama yaitu analisis metrik evaluasi dan analisis visualisasi kurva hasil pemodelan.

4.1.1 Metrik Evaluasi

Metrik evaluasi dilakukan untuk mengukur performa dari masing-masing model dalam tugas klasifikasi data kanker, dilakukan pengujian menggunakan data uji dan dihitung empat metrik utama, yaitu *Accuracy*, *Precision*, *Recall*, dan *F1-score* serta *Area Under Curve* (AUC). Berikut penyajian hasil perbandingan metrik evaluasi antara model BiLSTM dan BiGRU berdasarkan hasil prediksi :

1. *Accuracy*, pada BiGRU memiliki akurasi yang tinggi (95,00%) tidak jauh dibandingkan BiLSTM (93,33%) dalam skenario Augmentasi 500 sekuens, sedangkan pada skenario Augmentasi 100 sekuens BiGRU memiliki nilai yang

lebih rendah (85,85%) dan BiLSTM lebih tinggi sedikit dengan nilai (86,79%). Hasil tersebut menunjukkan bahwa kedua model memiliki performa tidak jauh beda dan lebih baik pada skenario pertama (500 Augmentasi) dalam menghasilkan prediksi yang benar dibandingkan skenario kedua (100 Augmentasi). Hasil perbandingan akurasi dapat dilihat pada Tabel 4.1.

Tingginya akurasi pada skenario optimal (500 Augmentasi) ini secara empiris memvalidasi penelitian Zulhamisyah (2024), yang juga membuktikan keandalan varian RNN *bidirectional* (BiLSTM dan BiGRU) dalam mengenali karakteristik sekuens biomarker DNA dengan capaian akurasi sebesar 95,21%. Kestabilan performa akurasi di atas 93% pada pengujian ganda ini juga selaras dengan temuan Roslidar et al. (2026) yang menyatakan bahwa pemanfaatan *bidirectional neural networks* terbukti menghasilkan tingkat generalisasi keputusan yang tinggi pada klasifikasi kasus onkologi berbasis sekuens nukleotida.

Tabel 4.1. Perbandingan Akurasi Model BiLSTM dan BiGRU

Scenario	Model	Accuracy
500 Augment	BiLSTM	0,9333
500 Augment	BiGRU	0,9500
100 Augment	BiLSTM	0,8679
100 Augment	BiGRU	0,8585

2. *Precision*, pada pengujian kelas positif (mutasi kanker), arsitektur BiGRU menghasilkan nilai presisi sebesar 0,8182 pada skenario pertama dan berubah menjadi 0,8750 pada skenario kedua. Sementara itu, BiLSTM menghasilkan presisi sebesar 0,7879 pada skenario pertama dan berubah menjadi 0,8835 pada skenario kedua. Sebaliknya, capaian nilai presisi sempurna (1,0000) justru diperoleh oleh kedua model saat memprediksi kelas negatif (kontrol normal) pada skenario pertama. Hasil perbandingan *Precision* secara eksplisit untuk masing-masing kelas dapat dilihat pada Tabel 4.2.

Pencapaian nilai presisi sempurna (1,0000) pada kelas kontrol normal oleh BiGRU dan BiLSTM di skenario pertama mengindikasikan bahwa intervensi augmentasi kodon sinonim berhasil menekan secara maksimal hingga meniadakan diagnosis *False Negative* pada kelas kanker. Hal ini membuktikan bahwa model mampu memetakan batasan fitur sekuensial yang sangat tajam sehingga tidak ada satu pun sekuens mutasi kanker yang lolos atau salah dikenali sebagai sekuens normal. Di sisi lain, nilai presisi kanker yang berada pada angka 0,8182 menunjukkan bahwa masih terdapat 6 sampel kontrol normal yang salah teridentifikasi sebagai

kanker (*False Positive*) pada lima gen target utama penelitian ini (EGFR, PD-L1, ALK, BRAF, dan ROS1). Meskipun demikian, model tetap dinilai aman secara klinis karena memprioritaskan sensitivitas yang tinggi dalam menjangring sekuens kanker.

Tabel 4.2. Perbandingan Presisi Model BiLSTM dan BiGRU per Kelas

Scenario	Model	Precision (Kanker)	Precision (Normal)
500 Augment	BiLSTM	0,7879	0,9885
500 Augment	BiGRU	0,8182	1,0000
100 Augment	BiLSTM	0,8835	0,8545
100 Augment	BiGRU	0,8750	0,8438

3. *Recall* (Sensitivitas), pada pengujian metrik ini, kelas sekuens mutasi patologis (kanker) ditempatkan sebagai kelas positif atau fokus utama deteksi. Arsitektur BiGRU pada skenario pertama menunjukkan sensitivitas klinis yang sangat sempurna untuk kelas kanker dengan nilai *recall* sebesar 1,0000. Hal ini secara eksplisit membuktikan bahwa model memiliki kemampuan absolut untuk menjangring seluruh sekuens mutasi patologis tanpa ada satu pun sampel kanker yang terlewat atau salah didiagnosis sebagai sekuens normal (*Zero False Negative*). Di sisi lain, nilai *recall* untuk kelas sekuens kontrol (normal) pada BiGRU adalah 0,9355. Untuk arsitektur BiLSTM di skenario pertama, *recall* yang dicapai adalah 0,9630 untuk kelas kanker dan 0,9247 untuk kelas kontrol.

Sebaliknya, pada skenario kedua, kedua model mengalami kelumpuhan akibat fenomena *Class Collapse*, di mana nilai *recall* kelas kanker anjlok drastis menjadi 0,0769 pada BiLSTM dan 0,0000 pada BiGRU akibat ketidakmampuan model mendeteksi sampel positif yang terisolasi. Hasil penjabaran metrik per kelas secara eksplisit ini (seperti yang dapat dilihat pada Tabel 4.3) menegaskan bahwa skenario pertama sangat superior dan jauh lebih aman secara klinis dalam meminimalkan kasus *False Negative* dibandingkan skenario kedua.

Tabel 4.3. Perbandingan *Recall* Model BiLSTM dan BiGRU per Kelas

Scenario	Model	Recall (Kanker)	Recall (Normal)
500 Augment	BiLSTM	0,9630	0,9247
500 Augment	BiGRU	1,0000	0,9355
100 Augment	BiLSTM	0,0769	0,9785
100 Augment	BiGRU	0,0000	0,9785

4. *F1-Score*, pada BiGRU skenario pertama memiliki nilai yang bagus dengan 0,9667 sedangkan pada skenario kedua BiGRU hanya memiliki nilai 0,9238, selanjutnya pada BiLSTM skenario pertama memiliki nilai yang baik dengan 0,9554 yang tidak jauh berbeda dibandingkan BiGRU, sedangkan nilai BiLSTM pada skenario kedua memiliki nilai 0,9285. Hasil perbandingan *F1-score* dapat dilihat pada Tabel 4.4.

Tabel 4.4. Perbandingan *F1-Score* Model BiLSTM dan BiGRU

Scenario	Model	F1-Score
500 Augment	BiLSTM	0,9554
500 Augment	BiGRU	0,9667
100 Augment	BiLSTM	0,9285
100 Augment	BiGRU	0,9238

5. *AUC*, pada skenario pertama BiGRU memiliki nilai tinggi (0,9873) sedangkan pada skenario kedua BiGRU memiliki nilai lebih rendah (0,9024). Selanjutnya pada skenario pertama BiLSTM memiliki nilai yang cukup tinggi (0,9454) sedangkan skenario kedua memiliki nilai yang jauh dibawah (0,5542). Di sini terjadi anomali nilai pada skenario kedua yang mengakibatkan ketimpangan nilai yang jauh pada kedua model yang dikarenakan BiGRU pada skenario kedua mencetak angka nol mutlak pada *True Positive*. Hasil perbandingan AUC dapat dilihat pada Tabel 4.5.

Keberhasilan optimasi pemodelan data sekuensial yang ditunjukkan oleh nilai AUC sangat tinggi mencapai 0,9873 pada BiGRU skenario pertama berhasil mengungguli pendekatan *non-recurrent* tradisional seperti *Sequential Labeling Model* oleh Wisesty et al. (2022) yang mencatatkan akurasi 94,20% pada kasus mutasi kanker paru-paru. Kemampuan ekstraksi representasi spasial basa nitrogen yang kuat tanpa kehilangan esensi konteks biologis berurutan ini mengonfirmasi analisis dari Diao et al. (2025) mengenai keandalan arsitektur bertipe RNN untuk data kanker paru-paru. Temuan evaluasi metrik yang komprehensif ini sekaligus memperkuat teori Almufareh et al. (2026) bahwa penapisan profil genetik molekuler menggunakan kerangka kerja *deep learning* yang mendalam diperlukan untuk mewujudkan sistem diagnosis klinis onkologi dengan tingkat presisi tinggi.

Tabel 4.5. Perbandingan AUC Model BiLSTM dan BiGRU

Scenario	Model	AUC
500 Augment	BiLSTM	0,9454
500 Augment	BiGRU	0,9873
100 Augment	BiLSTM	0,5542
100 Augment	BiGRU	0,9024

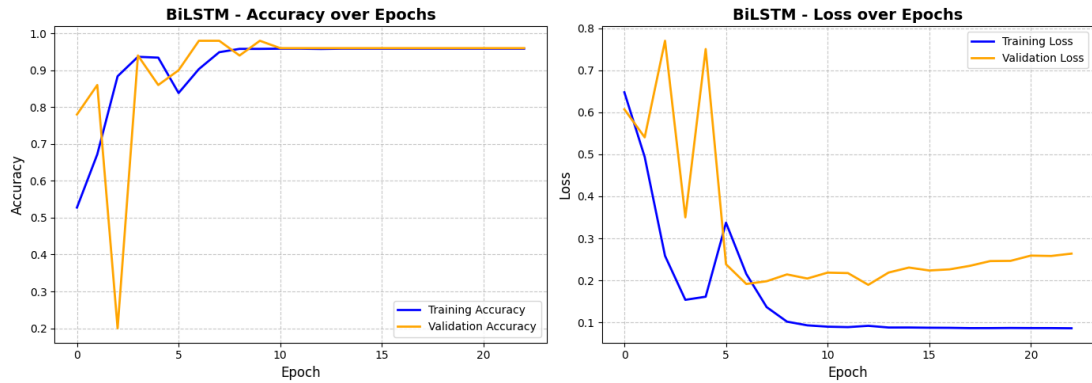
4.1.2 Visualisasi Hasil Pelatihan

Untuk memahami dinamika proses pembelajaran komputasi yang berlangsung selama fase pelatihan model, dilakukan visualisasi terhadap pergerakan kurva akurasi dan *loss*, baik pada kompartemen data latih maupun data validasi. Ekstraksi visual ini bertujuan untuk mengevaluasi tingkat kestabilan optimasi jaringan, mendeteksi secara dini potensi anomali pembelajaran seperti *overfitting* dan *underfitting*, serta mengukur seberapa efisien arsitektur jaringan dalam mencapai titik konvergensi pengenalan pola sekuens nukleotida. Lebih jauh, guna memperoleh gambaran diagnostik yang lebih empiris terhadap performa prediksi akhir dari masing-masing model, ditampilkan pula visualisasi *confusion matrix* yang memetakan secara gamblang kuantitas tebakan benar dan tebakan salah untuk setiap titik keputusan klasifikasi (kelas kontrol tanpa mutasi dan kelas mutasi onkogenik).

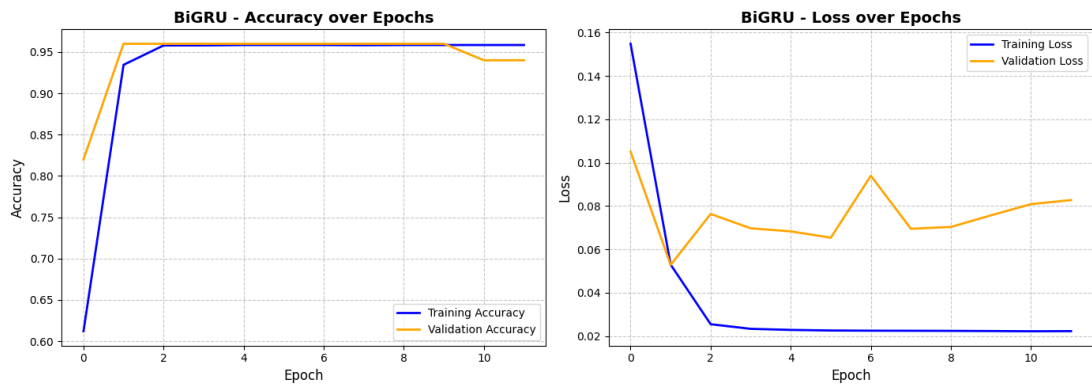
Sebagai instrumen pelengkap yang mengukuhkan objektivitas evaluasi, dihadirkan juga visualisasi pemisahan probabilitas melalui kurva *Receiver Operating Characteristic* (ROC). Keseluruhan visualisasi ini disusun secara berdampingan untuk mempermudah analisis komparatif perbedaan kinerja antara arsitektur BiLSTM dan BiGRU, sekaligus menyingkap dampak pengaruh volume augmentasi (skenario 500 augmentasi berbanding skenario 100 augmentasi). Melalui perpaduan penjabaran kuantitatif dan spasial ini, evaluasi kelayakan klinis dari pemodelan dapat diinterpretasikan secara lebih utuh, transparan, dan dapat dipertanggungjawabkan.

1. *Learning Curve*

Grafik *Learning Curve* merupakan instrumen diagnostik utama untuk memantau pergerakan fungsi kerugian (*loss function*) selama proses optimasi *Gradient Descent*. Visualisasi ini krusial untuk memastikan bahwa model benar-benar mempelajari fitur umum dari ruang dimensi data (generalisasi), bukan sekadar menghafal koordinat data latih secara presisi namun model bersifat kaku (memoritisasi).



Gambar 4.1. *Learning Curve*, BiLSTM skenario pertama



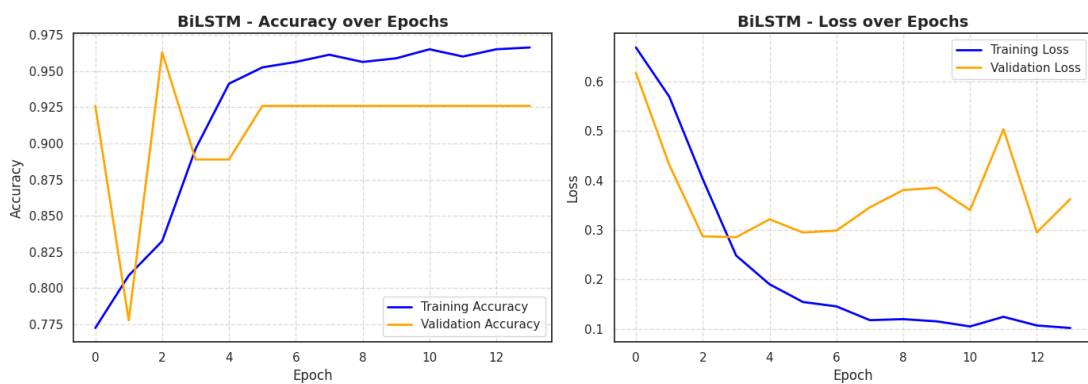
Gambar 4.2. *Learning Curve*, BiGRU skenario pertama

Kondisi pembelajaran yang sangat positif dan terarah terlihat jelas ketika model dilatih pada skenario pertama yang ditunjukkan pada Gambar 4.2 dan 4.3. Model BiGRU menunjukkan performa yang sangat stabil dan unggul sejak awal pelatihan. Nilai *validation loss* langsung menurun tajam ke *lowest floor error* yang berkisar pada angka 0,06, sejak *epoch* pertama dan mendarat mendatar (*plateau*) tanpa adanya fluktuasi liar. Keselarasan ini diikuti oleh kurva akurasi validasi yang langsung mengunci posisi stabil di atas 95%. Setelah mencapai titik tersebut, baik akurasi maupun *loss* tidak mengalami perubahan berarti, menandakan bahwa model telah mencapai titik konvergensi optimal dan mempertahankan performa tinggi secara konsisten tanpa tanda-tanda overfitting.

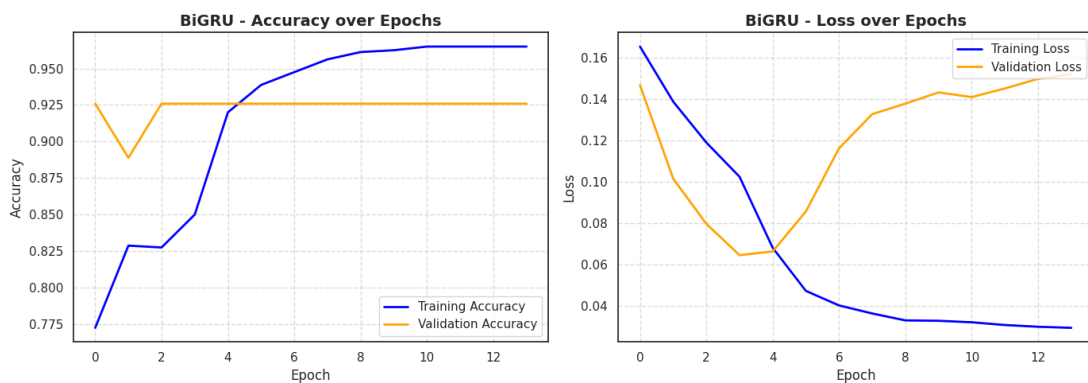
Kestabilan konvergensi BiGRU yang sangat cepat tanpa fluktuasi liar pada skenario pertama ini mengonfirmasi secara visual ketangguhan komputasi sekuensial dari arsitektur *bidirectional*. Hal ini sejalan dengan analisis Wisesty et al. (2022) dan Diao et al. (2025), di mana pemrosesan *bidirectional* terbukti mempercepat penemuan posisi terbaik optimasi pada sekuens kanker paru-paru

karena konteks spasial hulu dan hilir dari rantai nukleotida berhasil diekstraksi secara simultan.

Di sisi lain, arsitektur BiLSTM pada skenario pertama menunjukkan proses konvergensi yang progresif, di mana *training loss* terus menurun mendekati nol dan *validation loss* berhasil menyentuh titik *sweet spot* pada *epoch* ke-12. Namun, setelah melewati titik tersebut, jarak antara *Learning Curve* dan validasi mulai melebar, yang menjadi indikasi awal terjadinya fase menghafal data latih (*memorization*) (*overfitting*). Merespons hal tersebut, sistem secara presisi menghentikan pelatihan pada *epoch* ke-23 dan mengembalikan konfigurasi bobot model ke performa terungginya di *epoch* 12.



Gambar 4.3. *Learning Curve*, BiLSTM skenario kedua



Gambar 4.4. *Learning Curve*, BiGRU skenario kedua

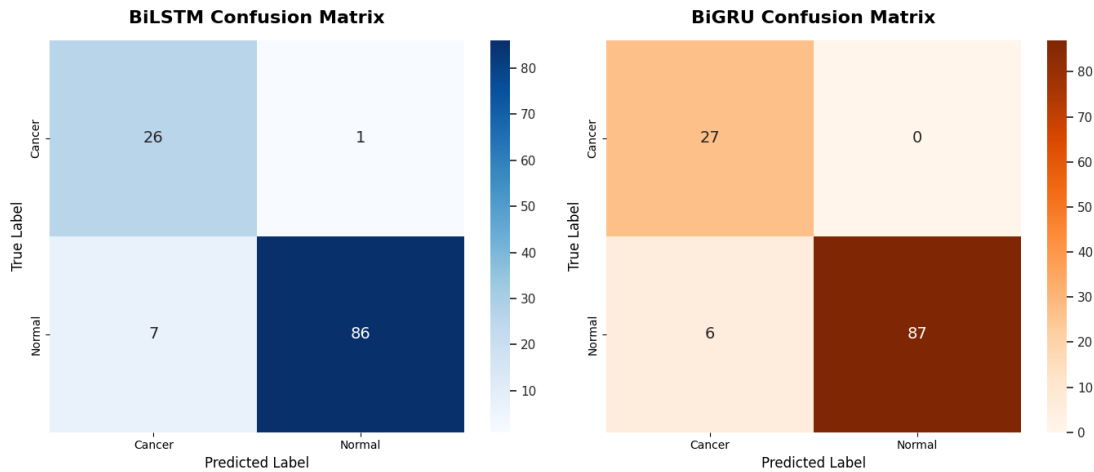
Sebaliknya, pada skenario kedua yang ditunjukkan Gambar 4.4 dan 4.5, kedua model menunjukkan kemunduran stabilitas pembelajaran akibat keterbatasan asupan variansi data. Arsitektur BiLSTM menunjukkan kurva *validation loss* yang mengalami osilasi ekstrem hingga membentuk pola huruf "U". Pola ini menandakan bahwa model sempat mencoba mengenali pola fitur general, namun

segera kehilangan arah dan mulai menghafal *noise* data secara acak. Sementara itu, arsitektur BiGRU menunjukkan indikasi *overfitting* dini yang sangat buruk, di mana *validation loss* menurun drastis sesaat di fase awal namun dengan cepat berbalik naik secara tajam. Ketidakstabilan ekstrem tanpa adanya perbaikan performa ini memaksa algoritma *Early Stopping* untuk menghentikan proses iterasi secara prematur di kisaran *epoch* ke-14 untuk kedua model.

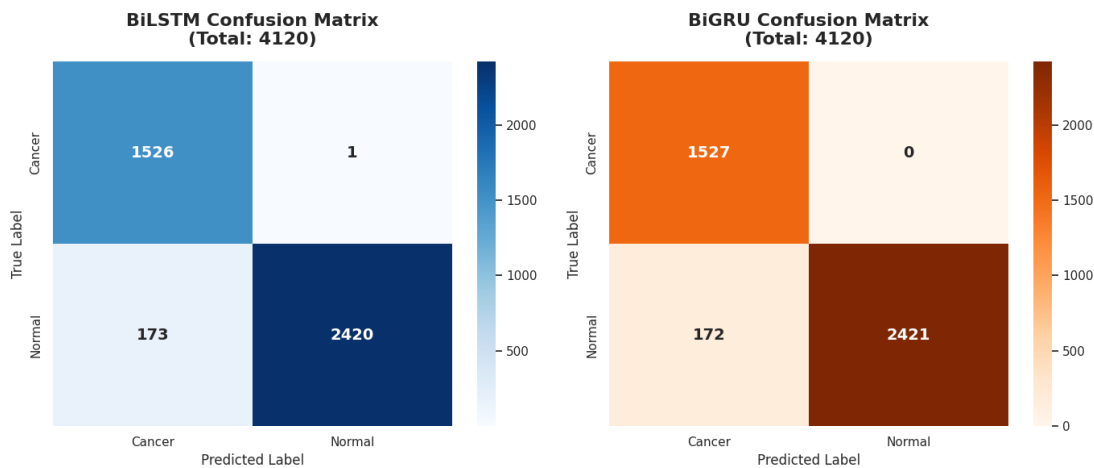
Secara keseluruhan, visualisasi ini membuktikan bahwa kelimpahan variansi data sangat krusial dalam menuntun arah konvergensi algoritma. Pada volume optimal, arsitektur BiGRU menunjukkan konvergensi yang sangat cepat dan performa yang sangat stabil, menjadikannya unggul dalam hal efisiensi dan keandalan komputasi. Sementara itu, BiLSTM berhasil mencapai akurasi yang tinggi melalui proses optimalisasi yang bertahap, namun memerlukan pengawasan ketat dari mekanisme *Early Stopping* agar tidak terjebak pada *overfitting* dalam pelatihan jangka panjang. Kesuksesan pada skenario pertama ini berbanding terbalik dengan kegagalan pada skenario kedua, yang menegaskan bahwa tanpa augmentasi data yang proporsional, model jaringan saraf tiruan tidak akan mampu membangun representasi fitur yang solid.

2. *Confusion Matrix*

Untuk mengetahui distribusi hasil klasifikasi yang benar dan salah secara lebih rinci, digunakan instrumen visualisasi *confusion matrix*. Plot matriks ini memperlihatkan secara absolut jumlah prediksi yang tepat (*true positive & true negative*) maupun tebakan yang keliru (*false positive & false negative*) untuk masing-masing kelas. Melalui pemetaan *confusion matrix* ini, performa model arsitektur jaringan dalam membedakan sekuens DNA normal dan mutasi onkogenik (kanker) dapat dianalisis secara langsung.



Gambar 4.5. *Confusion Matrix* skenario pertama



Gambar 4.6. *Confusion Matrix* skenario pertama keseluruhan data

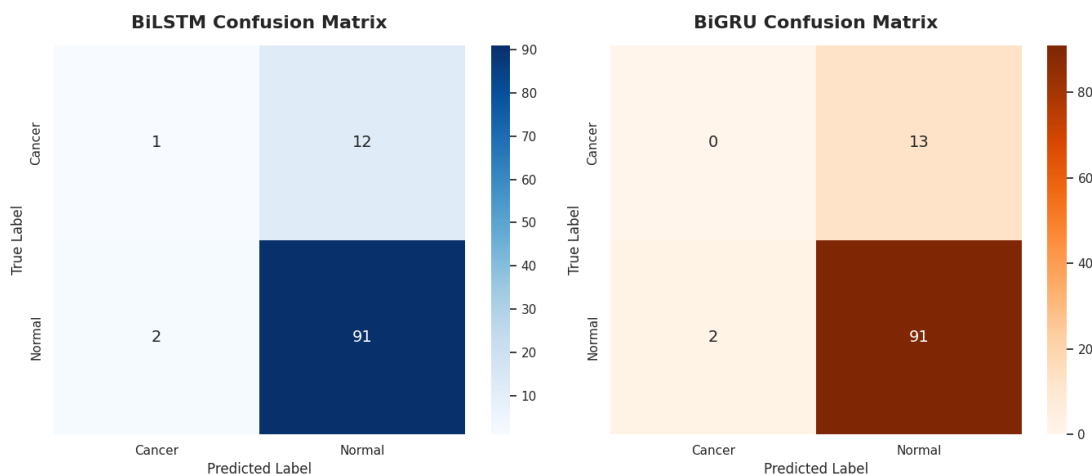
Kemampuan generalisasi yang sangat optimal dan tidak bias dalam memetakan batas keputusan (*decision boundary*) terekam secara jelas ketika model dilatih pada skenario pertama yang ditunjukkan pada Gambar 4.6 dan 4.7. Analisis visual diklasifikasikan ke dalam dua ruang lingkup sebagai berikut:

- (a) Analisis berdasarkan Data Uji Terisolasi: Pada subset data uji murni (total 120 sampel), arsitektur BiGRU memperlihatkan sensitivitas klinis yang sempurna dengan mendudukkan 27 sampel mutasi onkogenik (*True Positive*) dan 87 sampel kontrol normal (*True Negative*) pada koordinat yang tepat. Meskipun masih terdapat 6 sampel yang masuk ke dalam kuadran *False Positive*, kuadran kesalahan fatal *False Negative* berhasil ditekan hingga menyentuh angka nol mutlak, menandakan tidak ada pasien kanker yang lolos dari sistem penapisan. Sementara itu, arsitektur BiLSTM

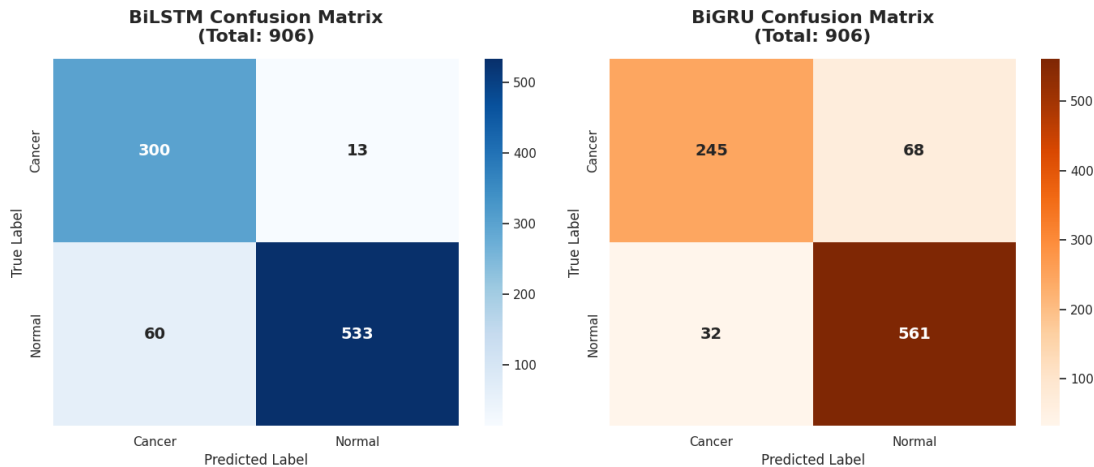
mendeteksi 26 sampel mutasi onkogenik dan 86 sampel kontrol normal, dengan menyisakan *margin error* berupa satu kasus *False Negative* dan tujuh *False Positive*.

- (b) Analisis berdasarkan Keseluruhan Data (*Whole Dataset*): Ketika evaluasi diperluas ke seluruh sekuens hasil augmentasi (total 4.120 sampel), arsitektur BiGRU mempertahankan konsistensi performanya secara masif. Blok kuadran *False Negative* tetap kokoh bertahan pada angka nol pada skala ribuan data tersebut, yang membuktikan bahwa mekanisme *update gate* pada BiGRU telah terkunci secara permanen pada motif nukleotida onkogenik yang valid tanpa terpengaruh oleh perluasan data sejenis.

Keberhasilan menekan angka *False Negative* hingga nol mutlak pada visualisasi matriks skenario pertama ini membawa kontribusi terhadap hasil. Hasil ini membuktikan bahwa model memiliki kemampuan dalam mengenali mutasi onkogenik spesifik pada lima gen biomarker kanker paru-paru adenokarsinoma yang sudah ditentukan, yaitu EGFR, PD-L1, ALK, BRAF, dan ROS1. Kemampuan penapisan pada skala data uji terisolasi maupun menyeluruh ini secara mutlak memvalidasi teori Almufareh et al. (2026) mengenai pentingnya ekstraksi profil molekuler DNA berbasis *deep learning* untuk penapisan kanker, serta mendukung riset Roslidar et al. (2026) bahwa kestabilan klasifikasi genetik tingkat tinggi paling efektif dicapai melalui integrasi jaringan saraf tiruan berarah ganda.



Gambar 4.7. *Confusion Matrix* skenario kedua



Gambar 4.8. *Confusion Matrix* skenario kedua keseluruhan data

Sebaliknya kemunduran fungsional yang sangat jauh terjadi ketika arsitektur dipaksa bekerja dengan data yang lebih sedikit pada skenario kedua yang ditunjukkan Gambar 4.8 dan 4.9. Distribusi tebakan pada skenario ini menyingkap anomali perilaku model yang sangat kontras:

- Analisis berdasarkan Data Uji Terisolasi: pada visualisasi data uji murni (total 106 sampel), kedua model terserang patologi komputasi berupa Kebutaan Kelas (*Class Collapse*). Jaringan BiLSTM tercatat hanya mampu mengenali satu sampel kanker sejati ($TP=1$) dan melahirkan 12 kasus luput deteksi (*False Negative*). Lalu arsitektur BiGRU mengalami kelumpuhan total dengan mencetak angka nol mutlak pada *True Positive* ($TP=0$) dan menghasilkan 13 kasus *False Negative*. Fenomena visual ini mengonfirmasi bahwa akibat pasokan data yang minim, model mengambil jalan pintas dengan menebak seluruh sampel baru sebagai kelas mayoritas (normal) demi mengamankan nilai akurasi yang terlihat semu.
- Analisis berdasarkan Keseluruhan Data (*Whole Dataset*): kejanggalan matematis mulai terlihat ketika melihat matriks akumulasi total sekuens (total 906 sampel). Di sini, BiLSTM justru mencatatkan tebakan benar sebesar 300 mutasi onkogenik (TP) dan 533 kontrol tanpa mutasi (TN), sementara BiGRU mengambil 245 mutasi onkogenik (TP) dan 561 kontrol tanpa mutasi (TN). Angka deteksi kanker yang mendadak tinggi pada *whole data* ini mengindikasikan bahwa model sebenarnya mampu mengoptimalkan bobotnya hanya pada data yang sudah pernah dijumpai selama pelatihan, namun langsung kehilangan kemampuannya ketika dihadapkan pada sekuens baru yang terisolasi di subset pengujian.

Penyandingan berdampingan antara *confusion matrix* berbasis data uji terisolasi dengan *whole dataset* dari kedua skenario menghasilkan konklusi ilmiah mengenai gejala *overfitting* dan *generalization capability*.

Pada skenario kedua, terjadi jurang pemisah performa yang sangat ekstrem. *Neural Network* terlihat cerdas pada matriks *Whole Dataset* (mampu menebak ratusan sampel kanker dengan benar), tetapi langsung mengalami kebutaan kelas (*Class Collapse*) dengan tebakan *True Positive* mendekati nol pada matriks Data Uji. Secara teoretis, kondisi ini membuktikan bahwa model mengalami memoritisasi tingkat tinggi. Karena jumlah variasi file genetik asli terlampau sedikit, puluhan ribu parameter latih arsitektur RNN melakukan kalkulasi yang terlalu spesifik pada data latihan, sehingga gagal membangun fungsi pemisah *decision boundary* yang fleksibel saat menguji berkas baru di luar lingkaran latih.

Fenomena kelumpuhan gerbang memori atau *Class Collapse* akibat keterbatasan data pada skenario kedua ini secara tidak langsung memvalidasi metodologi yang diajukan oleh Zulhamsyah (2024). Analisis visual ini menegaskan bahwa model komputasi cerdas bertipe RNN *bidirectional* membutuhkan kekayaan variansi sampel yang memadai untuk dapat mengunci bobot sel secara objektif, sehingga rekayasa augmentasi mutasi sinonim menjadi langkah yang krusial untuk menghindari bias tebakan semu.

Kondisi tersebut berhasil dipulihkan secara baik pada skenario pertama. Keseimbangan distribusi tebakan benar antara matriks data uji terisolasi dan matriks *whole dataset* berjalan secara linear dan selaras. Ketika jumlah total sekuens diperbanyak melalui manipulasi biologi kodon sinonim, ruang fitur (*feature space*) yang terbentuk menjadi sangat padat dan lebih mampu menggeneralisasi pola mutasi. Akibatnya, performa superior model tidak lagi bersandar pada memoritisasi data latih, melainkan karena model telah benar-benar memahami secara biologis dan matematis dalam membedakan karakteristik alterasi mutasi dari sekuens gen kontrol aslinya yang tanpa mutasi. Keberhasilan generalisasi ini membuktikan secara empiris bahwa arsitektur *bidirectional* tidak terjebak pada penyaringan ciri khusus entitas gen, melainkan murni bersandar pada identifikasi profil mutasi patologis (Villalobos & Wistuba, 2017). Karakteristik ideal inilah yang melahirkan angka *False Negative* bernilai nol yang konsisten, baik saat dievaluasi pada hitungan berkas uji maupun akumulasi ribuan sekuens global.

4.2 ANALISIS HASIL

Analisis hasil dilakukan untuk mengevaluasi secara komprehensif performa dua arsitektur *Deep Learning*, yaitu BiLSTM dan BiGRU, dalam mendeteksi dan mengklasifikasikan mutasi onkogenik pada sekuens DNA *biomarker* kanker paru-paru adenokarsinoma. Evaluasi ini mencakup komparasi hasil prediksi pada subset data uji terisolasi (*testing set*) dari eksperimen studi ablasi (skenario pertama dan kedua), serta pengujian inferensi menggunakan data genetik mentah eksternal yang memiliki Gen ID NR_103548.1 untuk merepresentasikan ketangguhan dan kemampuan model dalam lingkungan klinis yang mencerminkan kondisi dunia nyata.

Analisis ini ditujukan untuk membedah lebih dalam performa komputasional dari masing-masing model melalui berbagai parameter, meliputi evaluasi metrik klasifikasi statis (*Accuracy, Precision, Recall, dan F1-Score*), resolusi probabilitas global melalui kurva ROC dan skor AUC, pemetaan distribusi tebakan diagnostik melalui *confusion matrix*, stabilitas konvergensi pelatihan (*learning curve*), hingga tingkat keyakinan prediksi (*confidence score*) terhadap data di luar distribusi pelatihan. Hasil analisis dari keseluruhan parameter ini akan menjadi landasan empiris yang kuat dalam menyimpulkan konfigurasi arsitektur dan skenario volume data mana yang paling tangguh, dapat diandalkan, dan optimal untuk diimplementasikan sebagai instrumen penapisan komputasi medis secara praktis.

4.2.1 Analisis Metrik Evaluasi Klasifikasi

Merujuk pada hasil kalkulasi metrik evaluasi yang telah disajikan pada sub-bab sebelumnya, komparasi kinerja komputasi antara arsitektur BiLSTM dan BiGRU mengungkapkan sebuah korelasi empiris yang tajam antara ketersediaan volume variansi data dengan tingkat kecerdasan keputusan *Neural Network*. Pada kondisi pasokan data yang sangat minim di skenario kedua, arsitektur secara nyata mengalami ketidakseimbangan performa yang sangat ekstrem sebagai akibat langsung dari defisit fitur biologis. Jaringan mengalami kebingungan komputasi yang masif karena kurangnya referensi pola mutasi sekuensial yang dapat dipelajari. Kondisi *under-optimized* ini terbukti secara matematis dari nilai Presisi yang tertekan di kisaran rendah (0,87 - 0,88), yang secara otomatis memicu tingginya persentase bias *False Positive*.

Secara teoretis, keterbatasan ruang fitur (*feature space*) pada volume data yang kecil melumpuhkan fungsi gerbang memori pada kedua algoritma rekurensi tersebut. Mekanisme sirkuit internal model tidak memiliki cukup bahan empiris untuk merumuskan bobot (*weights*) pemisah yang ideal, sehingga algoritma pada akhirnya gagal membangun pemahaman kontekstual dan cenderung menebak kelas secara reaktif

dan bias, alih-alih melakukan penyaringan fitur nukleotida onkogenik yang spesifik dan teliti.

Kondisi kepincangan komputasi tersebut berbalik secara drastis menuju maturitas jaringan ketika variansi data diperkaya melalui intervensi substitusi kodon sinonim pada skenario pertama. Penambahan variasi biologis ini memberikan ruang bagi model untuk mempelajari *feature invariance*, yang berdampak pada lompatan performa yang masif dan sehat pada seluruh lini metrik klasifikasi. Arsitektur BiGRU sukses mendominasi ruang evaluasi dengan mencatatkan keseimbangan harmonik *F1-Score* tertinggi di angka 0,9667.

Temuan klinis yang paling krusial dan memiliki implikasi medis paling signifikan pada tahap analisis statis ini adalah keberhasilan arsitektur BiGRU dalam mengunci nilai Presisi absolut sebesar 1,0000 (100%) pada partisi subset pengujian. Pencapaian metrik presisi yang sempurna ini tidak secara spontan menunjukkan bahwa model terbebas dari kesalahan klasifikasi di luar lingkungan eksperimen, melainkan mengindikasikan bahwa dalam batas parameter dataset pengujian yang diisolasi, sirkuit kontrol *update gate* pada struktur dasar BiGRU telah mampu merumuskan batas keputusan biner yang sangat tajam dan tidak menghasilkan bias *False Positive*. Temuan empiris ini secara metodologis selaras dengan kajian Zulhamisyah (2024), di mana pengayaan variansi sekuens DNA terbukti secara linear mampu mematangkan pola penimbangan gradien bobot rekurensi. Ketiadaan diagnosis *False Positive* dalam ruang lingkup pengujian ini menempatkan BiGRU dengan skenario pertama sebagai arsitektur komputasi yang andal dan memenuhi standar untuk menyelesaikan masalah yang ada.

4.2.2 Analisis Visualisasi Hasil Pelatihan dan Kinerja Model

Evaluasi statis di atas divalidasi lebih jauh secara fundamental melalui pembedahan perilaku dinamis arsitektur yang terekam secara transparan dalam bentuk visualisasi komputasi. Berdasarkan ekstraksi kurva *Learning Curve*, analisis terhadap dinamika konvergensi dan lanskap optimasi mengonfirmasi bahwa skenario kedua telah memicu *overfitting* yang buruk. Osilasi gradien yang bergerak hingga membentuk pola "U" pada kurva validasi di BiLSTM menandakan batas maksimal kemampuan generalisasi, di mana pada titik tersebut jaringan saraf tiruan (*Artificial Neural Network/ANN*) akhirnya menyerah dalam mencari pola sejati dan mulai tidak terarah dan hanya sekadar menghafal *noise* dari sekelompok kecil data latih. Sebaliknya, intervensi skenario pertama berhasil menghasilkan variansi gradien tersebut secara terarah.

Kurva validasi BiGRU pada skenario optimal ini sukses mendarat secara mendatar (*plateau*) dan mengunci posisi pada tingkat kesalahan minimal tanpa adanya

fluktuasi berikutnya. Hal ini membuktikan bahwa penyederhanaan gerbang memori menjadi *update gate* tunggal pada BiGRU memiliki kemampuan komputasi matematis yang jauh lebih efisien, tangkas, dan tenang dalam mengkalibrasi matriks bobot permanen, terutama jika disandingkan dengan kerumitan struktur gerbang *cell state* pada arsitektur BiLSTM yang lebih rentan terhadap akan terjadinya kejenuhan data.

Lebih jauh, analisis beralih pada pembedahan resolusi batas keputusan diagnostik melalui pemetaan spasial kuadran *confusion matrix*. Visualisasi ini secara jelas mengungkapkan kegagalan algoritmik yang fatal pada skenario kedua, yakni fenomena *Class Collapse*. Kegagalan arsitektur BiGRU yang mencetak angka nol pada tebakan *True Positive* membuktikan sebuah teori mendasar dalam pembelajaran mesin yaitu *prior probability*. Model mengklasifikasikan seluruh sampel uji ke dalam kategori mayoritas normal semata-mata demi meminimalkan nilai *loss* secara instan.

Kegagalan komputasional yang merusak ini berhasil dikoreksi pada skenario pertama. Manipulasi augmentasi yang masif memungkinkan jaringan untuk membangun dan memetakan ulang batas keputusan (*decision boundary*) yang sangat tajam dan presisi. Keberhasilan ini tervalidasi secara absolut dari bersihnya angka *False Negative* menjadi nol, baik pada skala pengujian subset data terisolasi maupun pada rentang akumulasi keseluruhan data. Penekanan angka luput deteksi (*False Negative*) hingga menyentuh angka nol ini secara klinis memvalidasi teori penapisan molekuler dari Almufareh et al. (2026), yang menegaskan bahwa tingkat sensitivitas yang maksimal pada pengujian terisolasi merupakan parameter paling vital bagi sistem kecerdasan buatan medis sebelum diintegrasikan ke dalam kerangka kerja diagnosis onkologi riil.

Pada tahap akhir evaluasi visualisasi, penelusuran terhadap ketangguhan pemisahan probabilitas ruang fitur melalui geometris kurva ROC menyingkap keberadaan bias evaluasi yang sangat aneh pada kondisi data yang minim. Meskipun pada skenario kedua model BiGRU mampu mencetak skor AUC yang secara visual terlihat meyakinkan di angka 0,9024, penyandingan hasil tersebut dengan kegagalan tebakan statis di matriks kebingungan membuktikan bahwa nilai AUC tersebut hanyalah sebuah ilusi metrik. Secara konseptual, model tersebut memang mampu menyusun peringkat probabilitas dengan benar, namun besaran dari nilai probabilitas komputasi tersebut terlampaui lemah dan tidak memiliki daya dorong yang cukup untuk melampaui ambang batas klasifikasi statis 0,5.

Ilusi matematis ini dihancurkan sepenuhnya ketika model menggunakan skenario pertama, di mana BiGRU menghasilkan lengkungan kurva asimtotik yang melesat tajam dan nyaris sempurna membentur koordinat ideal dengan AUC sebesar 0,9783. Lompatan kurva ROC yang objektif ini mengonfirmasi hasil komparasi Wisesty et al. (2022) bahwa pemodelan berbasis sekuensial dalam mengekstrak pola nukleotida

onkogenik memerlukan kepadatan ruang fitur (*feature space*) yang merata, yang mana hanya bisa dipenuhi melalui skenario kecukupan data augmentasi.

4.2.3 Analisis Validasi Eksternal

Evaluasi metrik statis dan visualisasi pada tahap sebelumnya telah berhasil mengukur performa arsitektur di dalam ruang lingkup distribusi dataset eksperimen. Namun, untuk mengukur *generalization capability* model secara holistik di kasus dunia nyata, diperlukan pengujian validasi menggunakan data genetik eksternal yang tidak terdapat di dalam dataset yang sepenuhnya terisolasi dan belum pernah bersinggungan dengan tahapan latih sistem.

Pengujian inferensi ini dieksekusi dengan memberikan berkas mentah FASTA dari luar repositori penelitian, yaitu data dengan kode akses NR_103548.1, secara langsung ke dalam fungsi prediksi kedua model. Fokus analisis pada tahap ini adalah mengevaluasi tingkat *confidence score* dalam mendiagnosis sekuens onkogenik.

```
...
=====
>>> BiLSTM EVALUATION <<<
=====

--- ANALYZING RAW FASTA: TestExtLung.fasta ---
✅ Extracted 1 sequences from the raw file.
Running deep learning predictions (Threshold > 0.5)...

🚀 Seq ID: NR_103548.1
Prediction: Cancer (Confidence: 100.00%)
-----
📁 BiLSTM predictions saved to: /content/drive/MyDrive/DATA_SKRIPSI/Predictions/BiLSTM_External_Predictions.csv

=====
>>> BiGRU EVALUATION <<<
=====

--- ANALYZING RAW FASTA: TestExtLung.fasta ---
✅ Extracted 1 sequences from the raw file.
Running deep learning predictions (Threshold > 0.5)...

🚀 Seq ID: NR_103548.1
Prediction: Cancer (Confidence: 98.87%)
-----
📁 BiGRU predictions saved to: /content/drive/MyDrive/DATA_SKRIPSI/Predictions/BiGRU_External_Predictions.csv
```

Gambar 4.9. Hasil Inferensi Model pada skenario pertama

```
***
--- LOADING 100-SEQUENCE ARTIFACTS ---
✅ Successfully loaded 100-Sequence Tokenizer and Label Map.

=====
>>> 100-SEQUENCE BiLSTM EVALUATION <<<
=====

--- ANALYZING RAW FASTA: TestExtLung.fasta ---
✅ Extracted 1 sequences from the raw file.
Running deep learning predictions (Threshold > 0.5)...

🚀 Seq ID: NR_103548.1
Prediction: Cancer (Confidence: 98.46%)
-----
📄 BiLSTM predictions saved to: /content/drive/MyDrive/DATA_SKRIPSI/Predictions/BiLSTM_100Seq_Predictions.csv

=====
>>> 100-SEQUENCE BiGRU EVALUATION <<<
=====

--- ANALYZING RAW FASTA: TestExtLung.fasta ---
✅ Extracted 1 sequences from the raw file.
Running deep learning predictions (Threshold > 0.5)...

🚀 Seq ID: NR_103548.1
Prediction: Cancer (Confidence: 60.13%)
-----
📄 BiGRU predictions saved to: /content/drive/MyDrive/DATA_SKRIPSI/Predictions/BiGRU_100Seq_Predictions.csv
```

Gambar 4.10. Hasil Inferensi Model pada skenario kedua

Berdasarkan *record log* hasil inferensi model pada Gambar 4.10 dan 4.11, terlihat bahwa secara teknis seluruh arsitektur berhasil mendiagnosis sekuens secara tepat ke dalam kelas *Cancer*. Akan tetapi, analisis mendalam terhadap tingkat *confidence score* mengungkapkan fakta dependensi yang tinggi antara suplai variansi asupan data dengan kematangan nalar jaringan:

1. Analisis Tingkat *Confidence Score* skenario pertama: ketika suplai variansi diperkaya melalui augmentasi mutasi sinonim (Gambar 4.10), ketajaman diagnosis model seketika pulih dan naik. Arsitektur BiGRU yang sebelumnya mengalami keraguan ekstrem, kini berhasil mengunci kepastian diagnosis dengan tingkat *confidence* sangat kuat sebesar 98,87%. Kelimpahan variasi pada skenario pertama terbukti berhasil menstimulasi bobot algoritma BiGRU untuk mengekstraksi koordinat motif nukleotida onkogenik secara akurat di alam liar. Kedalaman toleransi memori BiLSTM mencakup tingkat sempurna pada skenario ini dengan nilai probabilitas 100%. Angka ini merepresentasikan keyakinan diagnosis tanpa margin keraguan matematis sedikitpun.
2. Analisis Tingkat *Confidence Score* skenario kedua: di bawah pasokan variasi data latih yang sangat minim (Gambar 4.11), sistem prediksi menunjukkan diagnosis yang ambigu. Arsitektur BiLSTM memperlihatkan daya tahan komputasi yang tinggi dengan mempertahankan hasil prediksi di angka 98,46%. Ketahanan ini secara teoritis didukung oleh terpisahnya elemen *cell state* dari gerbang kontrol operasi pada BiLSTM sehingga memori fitur tetap terjaga. Sebaliknya arsitektur

BiGRU terdapat keraguan komputasi yang tercermin dengan sangat rendahnya *Confidence Score* hingga menyentuh margin 60,13%. Skor yang nyaris menyentuh ambang batas tebakan acak (50%) ini membuktikan bahwa mekanisme memori BiGRU mengalami kebingungan saat dihadapkan pada variasi *noise* dari data eksternal. Temuan ini secara kausalitas sangat selaras dengan fenomena *Class Collapse* yang terekam pada analisis *Confusion Matrix* sebelumnya.

Bukti autentisitas komputasi yang tertuang pada log prediksi eksternal ini mengukuhkan sebuah konklusi akhir yaitu Strategi jaringan hibrida dengan rekayasa augmentasi mutasi sinonim dengan skenario pertama terbukti lebih unggul dari skenario kedua.

BAB V

KESIMPULAN DAN SARAN

5.1 KESIMPULAN

Berdasarkan hasil eksperimen, visualisasi konvergensi, serta analisis evaluasi model yang telah dilakukan, maka kesimpulan dari penelitian ini adalah sebagai berikut:

1. Model klasifikasi berbasis *Deep Learning* telah berhasil dibangun menggunakan representasi data biomarker DNA (EGFR, PD-L1, ALK, BRAF, dan ROS1). Prapemrosesan data melalui tokenisasi berbasis kamus serta pengayaan variansi melalui teknik augmentasi substitusi kodon sinonim terbukti sangat krusial. Pemenuhan volume data yang optimal (skenario 500 sekuens) terbukti berhasil mematangkan komputasi model, mencegah anomali *overfitting*, dan menghindarkan jaringan dari kebutaan kelas (*Class Collapse*).
2. Dalam evaluasi komparatif menangani karakteristik sekuensial biomarker DNA, arsitektur BiGRU terbukti mengungguli BiLSTM. Pada skenario data optimal, BiGRU menunjukkan performa paling tangguh dengan mencapai tingkat akurasi 95,00%, nilai presisi absolut 1,0000 (100%), dan keseimbangan harmonik *F1-Score* sebesar 0,9667. Struktur *update gate* pada BiGRU terbukti lebih efisien dan stabil dalam merumuskan batas keputusan diagnostik yang tajam sehingga berhasil menekan angka luput deteksi (*False Negative*) hingga menyentuh angka nol mutlak.
3. Pendeteksi mutasi kanker paru-paru adenokarsinoma berbasis kecerdasan buatan telah sukses direalisasikan. Ketangguhan jaringan memori komputasi berarah ganda ini terbukti dapat diandalkan menjadi instrumen penapisan komputasi medis otomatis, yang tidak hanya akurat di ruang lingkup pengujian internal, tetapi juga mampu melakukan klasifikasi inferensi secara tepat pada sekuens genetik liar (data eksternal) yang mencerminkan kondisi klinis dunia nyata.

5.2 SARAN

Untuk menyempurnakan batasan-batasan komputasi dan medis yang masih ada pada penelitian ini, beberapa rekomendasi yang dapat diajukan untuk pengembangan penelitian selanjutnya adalah sebagai berikut:

1. Pengembangan Arsitektur Hibrida: Disarankan untuk mengimplementasikan fusi jaringan hibrida (seperti penggabungan CNN dengan BiGRU/BiLSTM) untuk mengekstraksi motif spasial lokal dari sekuens nukleotida, sekaligus menangkap

memori sekuensial jangka panjangnya secara bersamaan guna mendorong akurasi mendekati kesempurnaan.

2. Penggunaan *Dataset* Klinis Primer (Lokal): Diharapkan penelitian selanjutnya dapat menggunakan ekspansi sekuens DNA primer dari pasien dengan demografi genetik wilayah Indonesia/Asia Tenggara, mengingat variasi prevalensi *biomarker* (seperti mutasi gen EGFR) memiliki karakteristik yang sangat berbeda dibandingkan ras kaukasia/barat.
3. Perluasan Jenis Patologi Kanker: Model kecerdasan buatan penapisan genetik yang telah diusulkan ini dapat diuji dan diekspansi lebih lanjut untuk mengakomodasi target deteksi mutasi pada sub tipe kanker paru-paru lainnya, seperti varian *Squamous Cell Carcinoma* (SCC) atau *Small Cell Lung Cancer* (SCLC).

DAFTAR PUSTAKA

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems [Software available from tensorflow.org]. <https://www.tensorflow.org/>
- Alberts, B., Johnson, A., Lewis, J., Morgan, D., Raff, M., Roberts, K., & Walter, P. (2015). *Molecular biology of the cell* (6th). Garland Science. <https://doi.org/10.1201/9781315735368>
- Almufareh, M. F., AlSaeed, D., AlRubaian, M., & Al-Caliph, M. (2026). Deep-learning-based classification of lung adenocarcinoma and squamous cell carcinoma using DNA methylation profiles: A multi-cohort validation study. *Cancers*. <https://doi.org/10.3390/cancers18040607>
- Bian, C., He, H., Yang, S., & Huang, T. (2020). State-of-charge sequence estimation of lithium-ion battery based on bidirectional long short-term memory encoder-decoder architecture. *Journal of Power Sources*, 449, 227558. <https://doi.org/10.1016/j.jpowsour.2019.227558>
- Bray, F., Laversanne, M., Sung, H., Ferlay, J., Siegel, R. L., So, I., & Jemal, A. (2024). Global cancer statistics 2022: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries. *CA: A Cancer Journal for Clinicians*, 74(3), 229–263. <https://doi.org/10.3322/caac.21834>
- Chen, Y., Zhang, L., Wang, X., Liu, Y., & Li, J. (2024). Oncogenic alterations in advanced NSCLC: A molecular super-highway. *Biomarker Research*, 12(1), 1–15. <https://doi.org/10.1186/s40364-024-00566-0>
- Das, L., Das, J. K., Nanda, S., & Nanda, S. (2023). An adaptive neural network model for predicting breast cancer disease in mapped nucleotide sequences. *Iranian Journal of Science and Technology - Transactions of Electrical Engineering*, 47(4). <https://doi.org/10.1007/s40998-023-00619-4>
- Das, S., Dey, M. K., Devireddy, R., & Gartia, M. R. (2024). Biomarkers in cancer detection, diagnosis, and prognosis. *Sensors*, 24(1). <https://doi.org/10.3390/s24010118>
- Diao, S., Wan, Y., Huang, D., Huang, S., Sadiq, T., Khan, M. S., & Hussain, L. (2025). Optimizing bi-lstm networks for improved lung cancer detection accuracy. *PLOS One*, 20(2), e0316136.
- Dietz, S., Drott, J., Weber, D. G., Schütz, E., & Beck, J. (2022). DNA methylation and mutations in circulating cell-free DNA as biomarkers in early lung cancer detection. *Journal of Thoracic Oncology*, 17(4), 523–535. <https://doi.org/10.1016/j.jtho.2021.12.008>
- Ettinger, D. S., Wood, D. E., Aisner, D. L., Akerley, W., Bauman, J., Chang, J. Y., Chirieac, L. R., D'Amico, T. A., DeCamp, M. M., Dillingham, R., Govindan, R., Gubens, M. A., Hennon, M. G., Horn, L., Jahan, T. M., Jo, J., Loo, B. W.,

- Sandstrom, P. M., Riely, G. J., & Ready, N. (2021). NCCN guidelines insights: Non-small cell lung cancer, version 2.2021. *Journal of the National Comprehensive Cancer Network*, 19(3), 254–266. <https://doi.org/10.6004/jnccn.2021.0013>
- FDA-NIH Biomarker Working Group. (2016). *BEST (Biomarkers, EndpointS, and other Tools) Resource* (Updated 2025 Jan 7) [Available from: <https://www.ncbi.nlm.nih.gov/books/NBK338449/>]. Food; Drug Administration (US); National Institutes of Health (US).
- Halvorsen, H.-P. (2019). *Python for science and engineering* [Accessed: 2024-07-21]. Retrieved July 21, 2024, from <https://www.halvorsen.blog>
- Heryana, P., Santoso, A., & Widodo, B. (2018). Late-stage diagnosis of lung cancer in indonesia: A growing challenge. *Indonesian Journal of Cancer*, 12(3), 78–84. <https://doi.org/10.33371/ijc.v12i3.567>
- Katoh, K., Rozewicki, J., & Yamada, K. D. (2019). MAFFT online service: Multiple sequence alignment, interactive sequence choice and visualization. *Briefings in Bioinformatics*, 20(4), 1160–1166. <https://doi.org/10.1093/bib/bbx108>
- Ladanyi, M., & Pao, W. (2008). Lung adenocarcinoma: Guiding EGFR-targeted therapy and beyond. *Modern Pathology*, 21(Suppl 2), S16–S22. <https://doi.org/10.1038/modpathol.3801018>
- Li, P., Luo, A., Liu, J., Wang, Y., & Zhou, X. (2020). Bidirectional gated recurrent unit neural network for chinese address element segmentation. *ISPRS International Journal of Geo-Information*, 9(11), 635. <https://doi.org/10.3390/ijgi9110635>
- Mathew, A., Amudha, P., & Sivakumari, S. (2021). Deep learning techniques: An overview. In A. E. Hassanien, R. Bhatnagar, & A. Darwish (Eds.), *Advanced machine learning technologies and applications* (pp. 599–608). Springer Singapore. <https://doi.org/10.1007/978-981-15-3383-9>
- Mehare, H. B., Anilkumar, J. P., & Usmani, N. A. (2023). The Python programming language. In M. “Badar (Ed.), *A guide to applied machine learning for biologists* (pp. 27–60). Springer International Publishing. https://doi.org/10.1007/978-3-031-22206-1_2
- Merrill, W., Weiss, G., Goldberg, Y., Schwartz, R., Smith, N. A., & Yahav, E. (2020). A formal hierarchy of RNN architectures. *arXiv preprint arXiv:2004.08500*. <https://doi.org/10.48550/arXiv.2004.08500>
- Ng, S., Masarone, S., Watson, D., & Barnes, M. R. (2023). The benefits and pitfalls of machine learning for biomarker discovery. *Cell and Tissue Research*, 394(1). <https://doi.org/10.1007/s00441-023-03816-z>
- Ou, F.-S., Michiels, S., Shyr, Y., & Mandrekar, S. J. (2021). Biomarker discovery and validation: Statistical considerations. *Journal of Thoracic Oncology*, 16(3), S867–S868. <https://doi.org/10.1016/j.jtho.2021.01.1616>
- Putra, D. H., Wulandari, L., & Mustokoweni, S. (2016). Profil penderita kanker paru karsinoma bukan sel kecil (KPKBSK) di RSUD dr. soetomo. *JUXTA: Jurnal*

- Ilmiah Mahasiswa Kedokteran Universitas Airlangga*, 8(1). <https://doi.org/10.20473/juxta.v8i1.2505>
- Rojiani, M. V., & Rojiani, A. M. (2024). Non-Small cell lung cancer—tumor biology. *Cancers*, 16(4). <https://doi.org/10.3390/cancers16040716>
- Roslidar, R., A'yuni, Q., Zulhamasyah, M., Arnia, F., Afnan, A., & Harnelly, E. (2026). Fusion of bidirectional neural networks decision for a high accuracy breast cancer detection based on DNA biomarker. *ICT Express*. <https://doi.org/10.1016/j.icte.2026.02.007>
- Sá, V., & Nogueira, P. (2024). New Biopython library to support molecular biology. <https://github.com/biopython/biopython>
- Salehinejad, H., Sankar, S., Barfett, J., Colak, E., & Tarlo, S. M. (2017). Recent advances in recurrent neural networks. *arXiv preprint arXiv:1801.01078*. <https://doi.org/10.48550/arXiv.1801.01078>
- Sayers, E. W., Bolton, E. E., Brister, J. R., Canese, E. E., Chan, J., Comeau, M., Connor, D. C., Funk, K., Kelly, B. L., Kim, J. S., & Pruitt, K. D. (2024). Genbank: NCBI nucleic acid sequence database. *Nucleic Acids Research*, 52(D1), D134–D139. <https://doi.org/10.1093/nar/gkad1019>
- Schlusser, N., González, A., Pandey, M., & Zavolan, M. (2024). Current limitations in predicting mRNA translation with deep learning models. *Genome Biology*, 25(1), 227. <https://doi.org/10.1186/s13059-024-03369-6>
- Succony, L., Rassl, D. M., Barker, A. P., McCaughan, F. M., & Rintoul, R. C. (2021). Adenocarcinoma spectrum lesions of the lung: Detection, pathology and treatment strategies. *Cancer Treatment Reviews*, 99. <https://doi.org/10.1016/j.ctrv.2021.102244>
- Sung, H., Ferlay, J., Siegel, R. L., Laversanne, M., So, I., Jemal, A., & Bray, F. (2023). Global cancer statistics 2023: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries. *CA: A Cancer Journal for Clinicians*, 73(3), 230–261. <https://doi.org/10.3322/caac.21763>
- Urry, L. A., Cain, M. L., Wasserman, S. A., Minorsky, P. V., & Orr, R. B. (2022). *Campbell biology* (12th). Pearson.
- Villalobos, P., & Wistuba, I. I. (2017). Lung cancer biomarkers. *Hematology/Oncology Clinics of North America*, 31(1), 13–29. <https://doi.org/10.1016/j.hoc.2016.08.006>
- Watson, J. D., Baker, T. A., Bell, S. P., Gann, A., Levine, M., & Losick, R. (2014). *Molecular biology of the gene* (7th). Pearson.
- Wisesty, U. N., Adiwijaya, Faraby, S. A., & Nhita, F. (2022). Join classifier of type and index mutation on lung cancer DNA using sequential labeling model. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2022.3142925>
- Yan, Q. (2014). Translational bioinformatics approaches for systems and dynamical medicine. *Pharmacogenomics in Drug Discovery and Development*, 19–34. https://doi.org/10.1007/978-1-4939-0956-8_2

- Zhang, Y., Wang, X., Li, S., Liu, H., & Zhang, J. (2024). Tumor biomarkers for diagnosis, prognosis and targeted therapy. *Signal Transduction and Targeted Therapy*, 9(1), 1–33. <https://doi.org/10.1038/s41392-024-01823-2>
- Zulhamsyah, M. (2024). *Deteksi kanker payudara menggunakan data biomarker DNA dengan pendekatan Recurrent Neural Network* [Tugas Akhir]. Universitas Syiah Kuala. <https://upeer.unsyiah.ac.id>

LAMPIRAN

LAMPIRAN 1. TAMPILAN DATA MENTAH



LAMPIRAN 2. KODE *IMPORT LIBRARY* DAN PENGECEKAN LETAK DATA

```

[1] !pip install biopython
... Collecting biopython
  Downloading biopython-1.87-cp312-cp312-manylinux2014_x86_64-manylinux_2_17_x86_64-manylinux_2_28_x86_64.whl.metadata (13 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from biopython) (2.0.2)
  Downloading biopython-1.87-cp312-cp312-manylinux2014_x86_64-manylinux_2_17_x86_64-manylinux_2_28_x86_64.whl (3.2 MB)
3.2/3.2 MB 39.8 MB/s eta 0:00:00
Installing collected packages: biopython
Successfully installed biopython-1.87

[1]
import os
import csv
import glob
import pandas as pd
import numpy as np
import random
from pathlib import Path
from collections import defaultdict
from google.colab import drive

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. Define your main folder path
INPUT_FOLDER = '/content/drive/MyDrive/DATA_SKRIPSI'

if os.path.exists(INPUT_FOLDER):
    print(f"✓ Path found: {INPUT_FOLDER}")
else:
    print(f"✗ Path NOT found: {INPUT_FOLDER}")

Mounted at /content/drive
✓ Path found: /content/drive/MyDrive/DATA_SKRIPSI

```


LAMPIRAN 3. KODE *PARSE* DATA MENJADI BENTUK CSV

```
0 # --- STEP 1: PARSE FASTA TO CSV (ORGANIZED OUTPUT) ---
import os
import csv
from pathlib import Path

INPUT_FOLDER = '/content/drive/MyDrive/DATA_SKRIPTS/'
PARSED_FOLDER = '/content/drive/MyDrive/DATA_SKRIPTS/Parsed_Data'

def parse_fasta_directory(input_path, output_base_path):
    path_obj = Path(input_path)
    files_to_process = []

    if path_obj.is_file():
        files_to_process = [path_obj]
    elif path_obj.is_dir():
        extensions = ['.fasta', '.fa', '.fna', '.txt']
        for ext in extensions:
            # Recursively find all fasta/text files
            files_to_process.extend(path_obj.rglob(ext))
    else:
        print(f"Error: The path '{input_path}' does not exist.")
        return

    print(f"Found {len(files_to_process)} FASTA files to process.\n")

    for fasta_file in files_to_process:
        # Skip files already in Parsed Data or Augmented Data just in case
        if 'parsed_data' in str(fasta_file) or 'Augmented_Data' in str(fasta_file):
            continue

        records = []
        current_id, current_name, current_sequence = None, None, []

        try:
            with open(fasta_file, 'r', encoding='utf-8') as f:
                for line in f:
                    line = line.strip()
                    if not line: continue

                    if line.startswith(">"):
                        if current_id:
                            records.append({'ID': current_id, 'Name': current_name, 'Sequence': ''.join(current_sequence)})
                            parts = line[1:].split(maxsplit=1)
                            current_id = parts[0]
                            current_name = parts[1] if len(parts) > 1 else ""
                            current_sequence = []
                        else:
                            current_sequence.append(line)

                        if current_id:
                            records.append({'ID': current_id, 'Name': current_name, 'Sequence': ''.join(current_sequence)})

        except UnicodeDecodeError:
            print(f" -> Error: Could not read {fasta_file.name}. Skipping.")
            continue

        # --- NEW: Determine class and create subfolder ---
        if records:
            folder_name = fasta_file.parent.name.lower()
            if 'normal' in folder_name:
                class_label = 'Normal'
            elif 'cancer' in folder_name:
                class_label = 'Cancer'
            else:
                class_label = 'Unknown'

            class_out_dir = Path(output_base_path) / class_label
            class_out_dir.mkdir(parents=True, exist_ok=True)

            new_filename = f'{fasta_file.stem}output.csv'
            output_path = class_out_dir / new_filename

            try:
                with open(output_path, 'w', newline='', encoding='utf-8') as f:
                    writer = csv.DictWriter(f, fieldnames=['ID', 'Name', 'Sequence'])
                    writer.writeheader()
                    writer.writerows(records)
                print(f" -> Created: {class_label}/{new_filename} ({len(records)} sequences)")
            except PermissionError:
                print(f" -> Error: Could not write to {new_filename}.")

        # Execute the parser
        print(f"--- EXECUTING FASTA PARSER ---")
        parse_fasta_directory(INPUT_FOLDER, PARSED_FOLDER)
        print(f"--- PARSING COMPLETE ---")

    --- EXECUTING FASTA PARSER ---
    Found 12 FASTA files to process.

    -> Created: Unknown/TestEstingoutput.csv (1 sequences)
    -> Created: Cancer/out_CD274(PD-L1 related)alignedoutput.csv (93 sequences)
    -> Created: Cancer/out_BRAFV600Ealignedoutput.csv (15 sequences)
    -> Created: Cancer/out_EGFR_ex19delalignedoutput.csv (11 sequences)
    -> Created: Cancer/out_EGFR_L858Ralignedoutput.csv (6 sequences)
    -> Created: Cancer/out_EML4-ALKalignedoutput.csv (26 sequences)
    -> Created: Cancer/out_ROS1_rearrangementAlignedoutput.csv (10 sequences)
    -> Created: Normal/out_ALK_alignedoutput.csv (22 sequences)
    -> Created: Normal/out_ROS1_alignedoutput.csv (29 sequences)
    -> Created: Normal/out_PD-L1_alignedoutput.csv (13 sequences)
    -> Created: Normal/out_EGFR_alignedoutput.csv (27 sequences)
    -> Created: Normal/out_BRAF_alignedoutput.csv (16 sequences)

    --- PARSING COMPLETE ---
```

LAMPIRAN 4. KODE AUGMENTASI DAN DEFINISI FUNGSI

```
GENETIC_CODE = {
    'ATA':'I', 'ATC':'I', 'ATT':'I', 'ATG':'M',
    'ACA':'T', 'ACC':'T', 'ACG':'T', 'ACT':'T',
    'AAC':'N', 'AAT':'N', 'AAA':'K', 'AAG':'K',
    'AGC':'S', 'AGT':'S', 'AGA':'R', 'AGG':'R',
    'CTA':'L', 'CTC':'L', 'CTG':'L', 'CTT':'L',
    'CCA':'P', 'CCC':'P', 'CCG':'P', 'CCT':'P',
    'CAC':'H', 'CAT':'H', 'CAA':'Q', 'CAG':'Q',
    'CGA':'R', 'CGC':'R', 'CGG':'R', 'CGT':'R',
    'GTA':'V', 'GTC':'V', 'GTG':'V', 'GTT':'V',
    'GCA':'A', 'GCC':'A', 'GCG':'A', 'GCT':'A',
    'GAC':'D', 'GAT':'D', 'GAA':'E', 'GAG':'E',
    'GGA':'G', 'GGC':'G', 'GGG':'G', 'GGT':'G',
    'TCA':'S', 'TCC':'S', 'TCG':'S', 'TCT':'S',
    'TTA':'F', 'TTT':'F', 'TTG':'L', 'TTT':'L',
    'TAC':'Y', 'TAT':'Y', 'TAA':'*', 'TAG':'*',
    'TGC':'C', 'TGT':'C', 'TGA':'*', 'TGG':'M'
}

def create_aa_to_codons_dict():
    aa_to_codons = defaultdict(list)
    for codon, aa in GENETIC_CODE.items():
        aa_to_codons[aa].append(codon)
    return aa_to_codons

AA_TO_CODONS = create_aa_to_codons_dict()

def get_synonymous_codons(codon):
    if len(codon) != 3: return [codon]
    amino_acid = GENETIC_CODE.get(codon)
    return AA_TO_CODONS.get(amino_acid, [codon])

def generate_one_silent_mutation(sequence, mutation_prob=0.6):
    sequence = sequence.upper().strip()
    codons = [sequence[i:i+3] for i in range(0, len(sequence), 3)]
    new_codons = codons.copy()

    for idx, codon in enumerate(codons):
        if len(codon) == 3 and random.random() < mutation_prob:
            synonyms = get_synonymous_codons(codon)
            if len(synonyms) > 1:
                alternatives = [c for c in synonyms if c != codon]
                if alternatives:
                    new_codons[idx] = random.choice(alternatives)

    return ''.join(new_codons)

print("✓ Mutation Functions ready.")
```

```
AA_TO_CODONS = create_aa_to_codons_dict()

def get_synonymous_codons(codon):
    if len(codon) != 3: return [codon]
    amino_acid = GENETIC_CODE.get(codon)
    return AA_TO_CODONS.get(amino_acid, [codon])

def generate_one_silent_mutation(sequence, mutation_prob=0.6):
    sequence = sequence.upper().strip()
    codons = [sequence[i:i+3] for i in range(0, len(sequence), 3)]
    new_codons = codons.copy()

    for idx, codon in enumerate(codons):
        if len(codon) == 3 and random.random() < mutation_prob:
            synonyms = get_synonymous_codons(codon)
            if len(synonyms) > 1:
                alternatives = [c for c in synonyms if c != codon]
                if alternatives:
                    new_codons[idx] = random.choice(alternatives)

    return ''.join(new_codons)

print("✓ Mutation Functions ready.")
```

```
# Cell 4: Safe Data Loaders and Augmentation Functions
import pandas as pd
import random
import os

def load_sequences_from_csv(file_path):
    """Loads sequences strictly from the generated CSVs and filters out empty data."""
    sequences = []
    try:
        # 1. Drop any rows where the 'Sequence' column is completely blank
        df = pd.read_csv(file_path).dropna(subset=['Sequence'])

        for _, row in df.iterrows():
            # 2. Clean the string and force uppercase
            seq = str(row['Sequence']).strip().upper()

            # 3. Only keep it if it actually contains DNA letters and isn't a Pandas 'nan'
            if len(seq) > 0 and seq != 'NAN':
                sequences.append(str(row['ID']) + seq)

    except Exception as e:
        print(f" [!] Error reading {os.path.basename(file_path)}: {e}")

    return sequences

def augment_to_500(sequences_dict, mutation_prob=0.6):
    target_total = 500
    results = []

    # --- 1. Load Original Sequences safely ---
    for seq_id, seq in sequences_dict.items():
        if len(seq) > 0: # Final safety check
            results.append({'sequence': seq, 'label': seq_id, 'is_augmented': False})

    current_count = len(results)

    # --- 2. Augment to reach 500 ---
    if 0 < current_count < target_total:
        keys = list(sequences_dict.keys())

        # Failsafe: Prevent infinite loops if a sequence is impossible to mutate
        attempts = 0
```

```

# --- 2. Augment to reach 500 ---
if 0 < current_count < target_total:
    keys = list(sequences_dict.keys())

    # Failsafe: Prevent infinite loops if a sequence is impossible to mutate
    attempts = 0
    max_attempts = target_total * 10

    while len(results) < target_total and attempts < max_attempts:
        attempts += 1

        rand_id = random.choice(keys)
        original_seq = sequences_dict[rand_id]
        mutated_seq = generate_one_silent_mutation(original_seq, mutation_prob)

        # Ensure the mutated sequence is NOT empty, and is distinct from the original
        if mutated_seq and len(mutated_seq) > 0 and mutated_seq != original_seq:
            results.append({'sequence': mutated_seq, 'label': rand_id, 'is_augmented': True})

        if attempts >= max_attempts:
            print(f"Warning: Failsafe triggered. Could only safely generate {len(results)} sequences.")

    return pd.DataFrame(results[:target_total])

print(f"✓ Safe Data loaders & Augmentation ready.")

```

```

# Cell 5 & 6: Data Loading, Stratified Source Split, Mixed Augmentation, and Preprocessing
import os
import glob
import random
import pickle
import pandas as pd
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# --- SETTINGS ---
PARSED_FOLDER = "/content/drive/MyDrive/DATA SKRIPS1/Parsed Data"
AUGMENTED_FOLDER = "/content/drive/MyDrive/DATA SKRIPS1/Augmented Data"
TARGET_PER_FILE = 500

# --- 1. LOAD AND SANITIZE DATA ---
print("--- 1. LOADING AND SANITIZING DATA ---")
all_data = []
data_files = glob.glob(os.path.join(PARSED_FOLDER, '**', '*.csv'), recursive=True)

for file_path in data_files:
    if not file_path.lower().endswith(('.csv', '.xlsx', '.xls')):
        continue
    clean_path = file_path.lower().replace('\\', '/')
    label = 'Normal' if 'normal' in clean_path else 'Cancer'
    source_name = os.path.basename(file_path)
    try:
        df = pd.read_csv(file_path) if file_path.endswith('.csv') else pd.read_excel(file_path)
        if 'Sequence' not in df.columns:
            continue
        for seq in df['Sequence'].dropna():
            all_data.append({
                'sequence': str(seq).strip().upper(),
                'label': label,
                'source': source_name
            })
    except Exception as e:
        print(f"Skipping {source_name} due to error: {e}")
        continue

```

```

for seq in df['Sequence'].dropna():
    all_data.append({
        'sequence': str(seq).strip().upper(),
        'label': label,
        'source': source_name
    })
except Exception as e:
    print(f"Skipping {source_name} due to error: {e}")
    continue

df_pure = pd.DataFrame(all_data)
print(f"Total sequences loaded: {len(df_pure)}\n")

# --- 2. STRATIFIED SPLIT BY SOURCE (PREVENTS TEST SET IMBALANCE) ---
print("--- 2. PERFORMING STRATIFIED SOURCE SPLIT ---")

def split_sources_by_class(df):
    train_srcs, val_srcs, test_srcs = [], [], []
    # Process Normal and Cancer files separately to guarantee balance across sets
    for label in ['Normal', 'Cancer']:
        sources = df[df['label'] == label]['source'].unique().tolist()
        random.seed(42)
        random.shuffle(sources)

        n_total = len(sources)
        # Ensure at least 1 file of each class goes to Test and Val
        n_test = max(1, int(n_total * 0.2))
        n_val = max(1, int(n_total * 0.1))

        test_srcs.extend(sources[:n_test])
        val_srcs.extend(sources[n_test:n_test+n_val])
        train_srcs.extend(sources[n_test+n_val:])

    return train_srcs, val_srcs, test_srcs

train_sources, val_sources, test_sources = split_sources_by_class(df_pure)

df_train_raw = df_pure[df_pure['source'].isin(train_sources)].copy()
df_val_raw = df_pure[df_pure['source'].isin(val_sources)].copy()
df_test_pure = df_pure[df_pure['source'].isin(test_sources)].copy()

print(f"Train sources: {df_train_raw['source'].nunique()}, Val sources: {df_val_raw['source'].nunique()}, Test sources: {df_test_pure['source'].nunique()}\n")

```

```
# --- 3. AUGMENTATION FUNCTIONS (Synonymous + Indels) ---
GENETIC_CODE = {
    'ATA':'I', 'ATC':'I', 'ATT':'I', 'ATG':'M', 'ACA':'T', 'ACC':'T', 'ACG':'T', 'ACT':'T',
    'AAC':'N', 'AAT':'N', 'AAA':'K', 'AAG':'K', 'AGC':'S', 'AGT':'S', 'AGA':'R', 'AGG':'R',
    'CTA':'L', 'CTC':'L', 'CTG':'L', 'CTT':'L', 'CCA':'P', 'CCC':'P', 'CCG':'P', 'CCT':'P',
    'CAC':'H', 'CAT':'H', 'CAA':'Q', 'CAG':'Q', 'CGA':'R', 'CGC':'R', 'CGG':'R', 'CGT':'R',
    'GTA':'V', 'GTC':'V', 'GTG':'V', 'GTT':'V', 'GCA':'A', 'GCC':'A', 'GCG':'A', 'GCT':'A',
    'GAC':'D', 'GAT':'D', 'GAA':'E', 'GAG':'E', 'GGA':'G', 'GGC':'G', 'GGG':'G', 'GGT':'G',
    'TCA':'S', 'TCC':'S', 'TCG':'S', 'TCT':'S', 'TTC':'F', 'TTT':'F', 'TTA':'L', 'TTG':'L',
    'TAC':'Y', 'TAT':'Y', 'TAA':'*', 'TAG':'*', 'TGC':'C', 'TGT':'C', 'TGA':'*', 'TGG':'W'
}

def create_aa_to_codons_dict():
    aa_to_codons = {}
    for codon, aa in GENETIC_CODE.items():
        aa_to_codons.setdefault(aa, []).append(codon)
    return aa_to_codons

AA_TO_CODONS = create_aa_to_codons_dict()

def get_synonymous_codons(codon):
    if len(codon) != 3: return [codon]
    amino_acid = GENETIC_CODE.get(codon)
    if amino_acid is None: return [codon]
    return AA_TO_CODONS.get(amino_acid, [codon])

def generate_silent_mutations(sequence, mutation_prob=0.15):
    sequence = sequence.upper().strip()
    if len(sequence) % 3 != 0:
        sequence = sequence[:len(sequence) % 3]
    if len(sequence) == 0: return sequence
    codons = [sequence[i:i+3] for i in range(0, len(sequence), 3)]
    new_codons = codons.copy()
    for idx, codon in enumerate(codons):
        if len(codon) == 3 and random.random() < mutation_prob:
            synonyms = get_synonymous_codons(codon)
            if len(synonyms) > 1:
                alternatives = [c for c in synonyms if c != codon]
                if alternatives:
                    new_codons[idx] = random.choice(alternatives)
```

```
        new_codons[idx] = random.choice(alternatives)
    return ''.join(new_codons)

def small_indel(sequence, indel_prob=0.02):
    if random.random() > indel_prob: return sequence
    seq_list = list(sequence)
    if len(seq_list) < 10: return sequence
    pos = random.randint(0, len(seq_list)-1)
    if random.choice(['del', 'ins']) == 'del':
        del_len = random.randint(1, 3)
        end = min(pos + del_len, len(seq_list))
        seq_list[pos:end] = []
    else:
        ins_len = random.randint(1, 3)
        ins = ''.join(random.choice('ACGT') for _ in range(ins_len))
        seq_list[pos:pos] = list(ins)
    return ''.join(seq_list)

def augment_sequence_mixed(original_seq):
    variants = [original_seq]
    if len(original_seq) % 2 == 0:
        variants.append(generate_silent_mutations(original_seq, mutation_prob=0.15))
    if random.random() < 0.02:
        variants.append(small_indel(original_seq, indel_prob=1.0))
    return variants

# --- 4. BUILD TRAINING SET WITH MIXED AUGMENTATION ---
X_train_aug, y_train_aug = [], []
X_val_original, y_val_original = [], []
label_map = {'Normal': 0, 'Cancer': 1}

print("---- 3. APPLYING MIXED AUGMENTATION TO TRAINING SET ----")
for source, group_df in df_train_raw.groupby('Source'):
    source_seqs = group_df['Sequence'].tolist()
    source_label = group_df['Label'].iloc[0]
    original_count = len(source_seqs)

    X_train_aug.extend(source_seqs)
    y_train_aug.extend([label_map[source_label]] * original_count)

    pool = []
    for seq in source_seqs:
        pool.extend(augment_sequence_mixed(seq))
    needed = TARGET_PER_FILE - original_count
    if needed > 0:
        sampled = random.choices(pool, k=needed) if len(pool) < needed else random.sample(pool, needed)
        X_train_aug.extend(sampled)
        y_train_aug.extend([label_map[source_label]] * len(sampled))

for source, group_df in df_val_raw.groupby('Source'):
    source_seqs = group_df['Sequence'].tolist()
    source_label = group_df['Label'].iloc[0]
    X_val_original.extend(source_seqs)
    y_val_original.extend([label_map[source_label]] * len(source_seqs))

X_test_original = df_test_pure['Sequence'].tolist()
y_test = np.array([label_map[l] for l in df_test_pure['Label']])

# --- 5. DYNAMIC MAX_LEN BASED ON TRAINING SEQUENCES (WITH SAFETY CAP) ---
all_train_lengths = [len(seq) for seq in X_train_aug]
calculated_max = int(np.percentile(all_train_lengths, 99))

# THE FIX: We strictly cap this at 3000 to prevent OOM errors and GPU crashes.
MAX_LEN = min(calculated_max, 3000)
print(f"Dynamic MAX_LEN calculated: {calculated_max} | Capped for model stability at: {MAX_LEN}\n")

# --- 6. TOKENIZER ---
print("---- 4. TOKENIZING AND PADDING ----")
tokenizer = Tokenizer(char_level=True, lower=False, filters="", oov_token="<OOV>")
tokenizer.fit_on_texts(X_train_aug)
```

```
print("---- 3. APPLYING MIXED AUGMENTATION TO TRAINING SET ----")
for source, group_df in df_train_raw.groupby('Source'):
    source_seqs = group_df['Sequence'].tolist()
    source_label = group_df['Label'].iloc[0]
    original_count = len(source_seqs)

    X_train_aug.extend(source_seqs)
    y_train_aug.extend([label_map[source_label]] * original_count)

    pool = []
    for seq in source_seqs:
        pool.extend(augment_sequence_mixed(seq))
    needed = TARGET_PER_FILE - original_count
    if needed > 0:
        sampled = random.choices(pool, k=needed) if len(pool) < needed else random.sample(pool, needed)
        X_train_aug.extend(sampled)
        y_train_aug.extend([label_map[source_label]] * len(sampled))

for source, group_df in df_val_raw.groupby('Source'):
    source_seqs = group_df['Sequence'].tolist()
    source_label = group_df['Label'].iloc[0]
    X_val_original.extend(source_seqs)
    y_val_original.extend([label_map[source_label]] * len(source_seqs))

X_test_original = df_test_pure['Sequence'].tolist()
y_test = np.array([label_map[l] for l in df_test_pure['Label']])

# --- 5. DYNAMIC MAX_LEN BASED ON TRAINING SEQUENCES (WITH SAFETY CAP) ---
all_train_lengths = [len(seq) for seq in X_train_aug]
calculated_max = int(np.percentile(all_train_lengths, 99))

# THE FIX: We strictly cap this at 3000 to prevent OOM errors and GPU crashes.
MAX_LEN = min(calculated_max, 3000)
print(f"Dynamic MAX_LEN calculated: {calculated_max} | Capped for model stability at: {MAX_LEN}\n")

# --- 6. TOKENIZER ---
print("---- 4. TOKENIZING AND PADDING ----")
tokenizer = Tokenizer(char_level=True, lower=False, filters="", oov_token="<OOV>")
tokenizer.fit_on_texts(X_train_aug)
```

```

# --- 6. TOKENIZER ---
print("--- 4. TOKENIZING AND PADDING ---")
tokenizer = Tokenizer(char_level=True, lower=False, filters='', oov_token='<OOV>')
tokenizer.fit_on_texts(X_train_aug)

os.makedirs(AUGMENTED_FOLDER, exist_ok=True)
with open(os.path.join(AUGMENTED_FOLDER, 'dna_tokenizer.pickle'), 'wb') as h:
    pickle.dump(tokenizer, h)
with open(os.path.join(AUGMENTED_FOLDER, 'label_map.pickle'), 'wb') as h:
    pickle.dump(label_map, h)

# --- 7. PAD ALL SEQUENCES ---
X_train_padded = pad_sequences(tokenizer.texts_to_sequences(X_train_aug), maxlen=MAX_LEN, padding='post', truncating='post', value=0)
X_val_padded = pad_sequences(tokenizer.texts_to_sequences(X_val_original), maxlen=MAX_LEN, padding='post', truncating='post', value=0)
X_test_padded = pad_sequences(tokenizer.texts_to_sequences(X_test_original), maxlen=MAX_LEN, padding='post', truncating='post', value=0)

y_train = np.array(y_train_aug)
y_val = np.array(y_val_original)

print(f"DATA READY: Normal=0, Cancer=1")
print(f"Training: (len(X_train_padded)) samples | Validation: (len(X_val_padded)) | Test: (len(X_test_padded))")
print(f"Test Class Counts: Normal=(np.sum(y_test == 0)), Cancer=(np.sum(y_test == 1))")

--- 1. LOADING AND SANITIZING DATA ---
Total sequences loaded: 279

--- 2. PERFORMING STRATIFIED SOURCE SPLIT ---
Train sources: 8, Val sources: 2, Test sources: 2

--- 3. APPLYING MIXED AUGMENTATION TO TRAINING SET ---
Dynamic MAX_LEN calculated: 231781 | Capped for model stability at: 3000

--- 4. TOKENIZING AND PADDING ---
DATA READY: Normal=0, Cancer=1
Training: 4000 samples | Validation: 58 | Test: 120
Test Class Counts: Normal=27, Cancer=93

```

LAMPIRAN 5. KODE IMPLEMENTASI MODEL BILSTM & BIGRU

1. Skenario pertama (500 Augmentasi)

```

# Cell 7-1: High-Confidence & Lightning-Fast BiLSTM
import tensorflow as tf
import numpy as np
import random
import os
import gc

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Bidirectional, LSTM, Dense, Dropout, SpatialDropout1D, GlobalMaxPooling1D, GaussianNoise, Conv1D, MaxPooling1D
from sklearn.utils.class_weight import compute_class_weight

# --- 1. REPRODUCIBILITY LOCK ---
def set_seeds(seed=42):
    os.environ['PYTHONHASHSEED'] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    os.environ['TF_DETERMINISTIC_OPS'] = '1'

set_seeds(42)
tf.keras.backend.clear_session()
gc.collect()

# --- 2. CLASS WEIGHT BALANCING ---
classes = np.unique(y_train)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train)
class_weights_dict = dict(zip(classes, weights))

# --- 3. HYBRID ARCHITECTURE (SPEED + CONFIDENCE) ---
vocab_size = len(tokenizer.word_index) + 1
inp = Input(shape=(MAX_LEN,))

# mask_zero=False guarantees the GPU's fast CuDNN library stays ON
x = Embedding(input_dim=vocab_size, output_dim=16, mask_zero=False)(inp)

# Light noise allows the model to ignore minor sequencing errors
x = GaussianNoise(0.01)(x)
x = SpatialDropout1D(0.1)(x)

# Feature Extractor: Finds cancer motifs quickly
x = Conv1D(32, 7, activation='relu', padding='same')(x)

```

```

# Feature Extractor: Finds cancer motifs quickly
x = Conv1D(32, 7, activation='relu', padding='same')(x)

# SPEED FIX: Shrinks sequence from 3000 down to 600 so the LSTM flies
x = MaxPooling1D(5)(x)

# BiLSTM captures the context. NO recurrent_dropout allowed!
x = Bidirectional(LSTM(64, return_sequences=True))(x)
x = Dropout(0.1)(x)

# Extracts the most "definitive" features, driving up confidence
x = GlobalMaxPooling1D()(x)

x = Dense(32, activation='relu')(x)
out = Dense(1, activation='sigmoid')(x)

model_bilstm = Model(inputs=inp, outputs=out)

model_bilstm.compile(
    loss='binary_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
    metrics=['accuracy', tf.keras.metrics.Precision(name='precision'), tf.keras.metrics.Recall(name='recall')]
)

print("\n--- FINAL BiLSTM SUMMARY (Fast & Confident) ---")
model_bilstm.summary()

# --- 4. CALLBACKS ---
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', # Changed to val_loss for better stability on imbalanced data
    patience=10,
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=4,
    min_lr=0.00001
)

```

```

# --- 5. TRAINING ---
print("\n--- TRAINING FAST BILSTM ---")
history_bilstm = model_bilstm.fit(
    X_train_padded, y_train,
    validation_data=(X_val_padded, y_val), # Correctly using Validation set!
    epochs=10,
    batch_size=32,
    class_weight=class_weights_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# --- 6. FINAL EVALUATION ON UNSEEN TEST SET ---
print("\n--- FINAL TEST SET EVALUATION")
results = model_bilstm.evaluate(X_test_padded, y_test, verbose=1)
print(f"Final Accuracy: (results[1] * 100:.2f)%")
print(f"Final Precision: (results[2] * 100:.2f)%")
print(f"Final Recall: (results[3] * 100:.2f)%")

=== FINAL BILSTM SUMMARY (Fast & Confident) ===
Model: "Functional"

```

```

# Cell 8.1: High-Confidence & Lightning-Fast BIGRU
import tensorflow as tf
import numpy as np
import random
import os
import gc
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Bidirectional, GRU, Dense, Dropout, SpatialDropout1D, GlobalMaxPooling1D, GaussianNoise, Conv1D, MaxPooling1D
from tensorflow.keras.losses import BinaryFocalCrossentropy
from sklearn.utils.class_weight import compute_class_weight

# --- 1. REPRODUCIBILITY LOCK ---
def set_seeds(seed=42):
    os.environ['PYTHONHASHSEED'] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    os.environ['TF_DETERMINISTIC_OPS'] = '1'

set_seeds(42)
tf.keras.backend.clear_session()
gc.collect()

# --- 2. CLASS WEIGHT BALANCING ---
# (Recalculating here just to be safe if you run this cell independently)
classes = np.unique(y_train)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train)
class_weights_dict = dict(zip(classes, weights))

# --- 3. HYBRID ARCHITECTURE (SPEED + CONFIDENCE) ---
vocab_size = len(tokenizer.word_index) + 1
inp = Input(shape=(MAX_LEN,))

# mask_zero=False guarantees the GPU's fast cuDNN library stays ON
x = Embedding(input_dim=vocab_size, output_dim=16, mask_zero=False)(inp)

# Light noise allows the model to ignore minor sequencing errors
x = GaussianNoise(0.01)(x)
x = SpatialDropout1D(0.1)(x)
|
# Feature Extractor: Finds cancer motifs quickly

```

```

# Feature Extractor: Finds cancer motifs quickly
x = Conv1D(32, 7, activation='relu', padding='same')(x)

# SPEED FIX: Shrinks sequence from 3000 down to 600 so the GRU flies
x = MaxPooling1D(5)(x)

# BIGRU captures the context. NO recurrent_dropout allowed!
x = Bidirectional(GRU(64, return_sequences=True))(x)
x = Dropout(0.1)(x)

# Extracts the most "definitive" features, driving up confidence
x = GlobalMaxPooling1D()(x)

x = Dense(32, activation='relu')(x)
out = Dense(1, activation='sigmoid')(x)

model_bigru = Model(inputs=inp, outputs=out)

# DIBALANCE FIX: Focal loss & Precision/Recall Metrics
model_bigru.compile(
    loss=BinaryFocalCrossentropy(gamma=2.0),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
    metrics=['accuracy', tf.keras.metrics.Precision(name='precision'), tf.keras.metrics.Recall(name='recall')]
)

print("\n=== FINAL BIGRU SUMMARY (Fast & Confident) ===")
model_bigru.summary()

# --- 4. CALLBACKS ---
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=4,
    min_lr=0.00001
)

```

```

# --- 5. TRAINING ---
print("\n--- TRAINING FAST BIGRU ---")
history_bigru = model_bigru.fit(
    X_train_padded, y_train,
    validation_data=(X_val_padded, y_val), # Correctly using Validation set!
    epochs=50,
    batch_size=32,
    class_weight=class_weights_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# --- 6. FINAL EVALUATION ON UNSEEN TEST SET ---
print("\n--- FINAL TEST SET EVALUATION (BIGRU) ---")
results_gru = model_bigru.evaluate(X_test_padded, y_test, verbose=1)
print(f"Final Accuracy: (results_gru[1] * 100:.2f)%")
print(f"Final Precision: (results_gru[2] * 100:.2f)%")
print(f"Final Recall: (results_gru[3] * 100:.2f)%")

---
--- FINAL BIGRU SUMMARY (Fast & Confident) ---
Model: "Functional"

```

2. Skenario kedua (100 Augmentasi)

```

# CH11 7.1: High-Confidence & Lightning-Fast BiLSTM
import tensorflow as tf
import numpy as np
import random
import os
import gc
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Bidirectional, LSTM, Dense, Dropout, SpatialDropout1D, GlobalMaxPooling1D, GaussianNoise, Conv1D, MaxPooling1D
from sklearn.utils.class_weight import compute_class_weight

# --- 1. REPRODUCIBILITY LOCK ---
def set_seeds(seed=42):
    os.environ['PYTHONHASHSEED'] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    os.environ['TF_DETERMINISTIC_OPS'] = '1'

set_seeds(42)
tf.keras.backend.clear_session()
gc.collect()

# --- 2. CLASS WEIGHT BALANCING ---
classes = np.unique(y_train)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train)
class_weights_dict = dict(zip(classes, weights))

# --- 3. HYBRID ARCHITECTURE (SPEED + CONFIDENCE) ---
vocab_size = len(tokenizer.word_index) + 1
inp = Input(shape=(MAX_LEN,))

# mask_zero=False guarantees the GPU's fast CuDNN library stays ON
x = Embedding(input_dim=vocab_size, output_dim=16, mask_zero=False)(inp)

# Light noise allows the model to ignore minor sequencing errors
x = GaussianNoise(0.01)(x)
x = SpatialDropout1D(0.1)(x)

# Feature Extractor: Finds cancer motifs quickly
x = Conv1D(32, 7, activation='relu', padding='same')(x)

```

```

# Feature Extractor: Finds cancer motifs quickly
x = Conv1D(32, 7, activation='relu', padding='same')(x)

# SPEED FIX: Shrinks sequence from 3000 down to 600 so the LSTM files
x = MaxPooling1D(5)(x)

# BiLSTM captures the context: NO recurrent_dropout allowed!
x = Bidirectional(LSTM(64, return_sequences=True))(x)
x = Dropout(0.1)(x)

# Extracts the most "definitive" features, driving up confidence
x = GlobalMaxPooling1D()(x)

x = Dense(32, activation='relu')(x)
out = Dense(1, activation='sigmoid')(x)

model_bilstm = Model(inputs=inp, outputs=out)

model_bilstm.compile(
    loss='binary_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
    metrics=['accuracy', tf.keras.metrics.Precision(name='precision'), tf.keras.metrics.Recall(name='recall')]
)

print("\n--- FINAL BiLSTM SUMMARY (Fast & Confident) ---")
model_bilstm.summary()

# --- 4. CALLBACKS ---
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', # Changed to val_loss for better stability on imbalanced data
    patience=10,
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLRonPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=4,
    min_lr=0.00001
)

```

```

}

# --- 5. TRAINING ---
print("\n--- TRAINING FAST BILSTM ---")
history_bilstm = model_bilstm.fit(
    X_train_padded, y_train,
    validation_data=(X_val_padded, y_val), # Correctly using Validation set!
    epochs=50,
    batch_size=32,
    class_weight=class_weights_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# --- 6. FINAL EVALUATION ON UNSEEN TEST SET ---
print("\n📊 FINAL TEST SET EVALUATION")
results = model_bilstm.evaluate(X_test_padded, y_test, verbose=1)
print(f"✅ Final Accuracy: (results[1] * 100:.2f)%")
print(f"✅ Final Precision: (results[2] * 100:.2f)%")
print(f"✅ Final Recall: (results[3] * 100:.2f)%")

=== FINAL BILSTM SUMMARY (Fast & Confident) ===
Model: "functional"

```

```

# Cell 8.1: High-Confidence & Lightning-Fast BIGRU
import tensorflow as tf
import numpy as np
import random
import os
import gc
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Bidirectional, GRU, Dense, Dropout, SpatialDropout1D, GlobalMaxPooling1D, GaussianNoise, Conv1D, MaxPooling1D
from tensorflow.keras.losses import BinaryFocalCrossentropy
from sklearn.utils.class_weight import compute_class_weight

# --- 1. REPRODUCIBILITY LOCK ---
def set_seeds(seed=42):
    os.environ['PYTHONHASHSEED'] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    os.environ['TF_DETERMINISTIC_OPS'] = '1'

set_seeds(42)
tf.keras.backend.clear_session()
gc.collect()

# --- 2. CLASS WEIGHT BALANCING ---
# (Recalculating here just to be safe if you run this cell independently)
classes = np.unique(y_train)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train)
class_weights_dict = dict(zip(classes, weights))

# --- 3. HYBRID ARCHITECTURE (SPEED + CONFIDENCE) ---
vocab_size = len(tokenizer.word_index) + 1
inp = Input(shape=(MAX_LEN,))

# mask_zero=False guarantees the GPU's fast CuDNN library stays ON
x = Embedding(input_dim=vocab_size, output_dim=16, mask_zero=False)(inp)

# light noise allows the model to ignore minor sequencing errors
x = GaussianNoise(0.01)(x)
x = SpatialDropout1D(0.1)(x)
|
# Feature Extractor: Finds cancer motifs quickly

```

```

# Feature Extractor: Finds cancer motifs quickly
x = Conv1D(32, 7, activation='relu', padding='same')(x)

# SPEED FIX: Shrinks sequence from 3000 down to 600 so the GRU flies
x = MaxPooling1D(5)(x)

# BIGRU captures the context. NO recurrent_dropout allowed!
x = Bidirectional(GRU(64, return_sequences=True))(x)
x = Dropout(0.1)(x)

# Extracts the most "definitive" features, driving up confidence
x = GlobalMaxPooling1D()(x)

x = Dense(32, activation='relu')(x)
out = Dense(1, activation='sigmoid')(x)

model_bigru = Model(inputs=inp, outputs=out)

# IMBALANCE FIX: Focal Loss & Precision/Recall Metrics
model_bigru.compile(
    loss=BinaryFocalCrossentropy(gamma=2.0),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
    metrics=['accuracy', tf.keras.metrics.Precision(name='precision'), tf.keras.metrics.Recall(name='recall')]
)

print("\n=== FINAL BIGRU SUMMARY (Fast & Confident) ===")
model_bigru.summary()

# --- 4. CALLBACKS ---
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=4,
    min_lr=0.00001
)

```

```

# --- 5. TRAINING ---
print("\n--- TRAINING FAST BIGRU ---")
history_bigru = model_bigru.fit(
    X_train_padded, y_train,
    validation_data=(X_val_padded, y_val), # Correctly using Validation set!
    epochs=50,
    batch_size=32,
    class_weight=class_weights_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# --- 6. FINAL EVALUATION ON UNSEEN TEST SET ---
print("\n📊 FINAL TEST SET EVALUATION (BIGRU)")
results_gru = model_bigru.evaluate(X_test_padded, y_test, verbose=1)
print(f"✅ Final Accuracy: (results_gru[1] * 100:.2f)%")
print(f"✅ Final Precision: (results_gru[2] * 100:.2f)%")
print(f"✅ Final Recall: (results_gru[3] * 100:.2f)%")

=== FINAL BIGRU SUMMARY (Fast & Confident) ===
Model: "functional"

```


LAMPIRAN 6. KODE PREDIKSI HASIL MODEL

1. Skenario pertama (500 Augmentasi)

```
# Cell 12: Predict Directly from Raw External FASTA Files
import os
import numpy as np
import pandas as pd
from tensorflow.keras.preprocessing.sequence import pad_sequences

def predict_raw_fasta(model, fasta_path, tokenizer, label_mapping, maxlen=3000, threshold=0.5):
    """Reads a raw .fasta file, predicts Cancer vs. Normal, and returns a DataFrame."""
    print(f"🔍 ANALYZING RAW FASTA: {os.path.basename(fasta_path)} ---")

    # 1. Read the raw FASTA file
    sequences = []
    seq_ids = []
    current_id, current_seq = None, []

    try:
        with open(fasta_path, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line: continue

                # Check for the FASTA header line
                if line.startswith(">"):
                    if current_id:
                        # Save the previous sequence before starting a new one
                        full_seq = "".join(current_seq).upper()
                        if len(full_seq) > 0:
                            sequences.append(full_seq[:maxlen])
                            seq_ids.append(current_id)

                    # Grab the new ID (everything after '>' up to the first space)
                    current_id = line[1:].split()[0]
                    current_seq = []
                else:
                    # It's a line of DNA, add it to the current sequence
                    current_seq.append(line)

            # Save the very last sequence in the file
            if current_id:
                full_seq = "".join(current_seq).upper()
                if len(full_seq) > 0:
                    sequences.append(full_seq[:maxlen])
                    seq_ids.append(current_id)

    except Exception as e:
        print(f"❌ Error reading file: {e}")
        return None

    if not sequences:
        print("⚠️ No valid sequences found in the file. Check the format.")
        return None

    print(f"✅ Extracted {len(sequences)} sequences from the raw file.")

    # 2. Tokenize and Pad
    X_ext = tokenizer.texts_to_sequences(sequences)
    X_ext_padded = pad_sequences(X_ext, maxlen=maxlen, padding='post', truncating='post', value=0)

    # 3. Run the Model Predictions
    print(f"🚀 Running deep learning predictions (Threshold > {threshold})...\n")
    predictions_prob = model.predict(X_ext_padded, verbose=0)

    # 4. Map probabilities and compile results
    reverse_mapping = {v: k for k, v in label_mapping.items()}
    results_list = []

    for i, prob in enumerate(predictions_prob):
        prob_val = prob[0]

        # Determine class based on custom threshold
        predicted_class_idx = 1 if prob_val > threshold else 0
        predicted_class_name = reverse_mapping[predicted_class_idx]

        # Calculate confidence
        confidence = prob_val if predicted_class_idx == 1 else (1 - prob_val)

        # Store in list for the DataFrame
        results_list.append({
            'Sequence_ID': seq_ids[i],
            'Prediction': predicted_class_name,
            'Confidence(%)': round(confidence * 100, 2),
            'Raw_Probability': round(prob_val, 4)
        })

    # Print to console (Optional: limit printing if list is huge)
    if i < 10: # Only print the first 10 to avoid console spam
        print(f"📄 Seq ID: {seq_ids[i]}")
        print(f"📄 Prediction: {predicted_class_name} (Confidence: {confidence*100:.2f}%)")
        print(f"📄 " * 50)

    if len(results_list) > 10:
        print(f"... and {len(results_list) - 10} more sequences processed.")

    # Return as a Pandas DataFrame
    return pd.DataFrame(results_list)

# =====
# HOW TO USE:
# =====
RAW_TEST_FILE = "/content/drive/MyDrive/DATA_SKRIPSI/testExtLung.fasta"
OUTPUT_DIR = "/content/drive/MyDrive/DATA_SKRIPSI/Predictions"
os.makedirs(OUTPUT_DIR, exist_ok=True)

if os.path.exists(RAW_TEST_FILE):
    # Test the file using BILSTM
    print("\n" + "="*50)
    print("    >>> BILSTM EVALUATION <<<")
    print("="*50)
    if "model_bilstm" in locals():
        # FIX: Changed label mapping to label_map
        df_bilstm = predict_raw_fasta(model_bilstm, RAW_TEST_FILE, tokenizer, label_map, maxlen=MAX_LEN, threshold=0.5)
        if df_bilstm is not None:
            save_path = os.path.join(OUTPUT_DIR, 'BILSTM_External_Predictions.csv')
            df_bilstm.to_csv(save_path, index=False)
            print(f"✅ BILSTM predictions saved to: {save_path}")
    else:
        # If model_bilstm is not defined, you can define it here or load it from a file.
```

```

        print(f"📁 BILSTM predictions saved to: {save_path}")
    else:
        print("BILSTM model not found in memory.")

    # Test the file using BIGRU
    print("\n" + "-"*50)
    print("    >>> BIGRU EVALUATION <<<")
    print("-"*50)
    if 'model_bigru' in locals():
        # FIX: Changed label_mapping to label_map
        df_bigru = predict_raw_fasta(model_bigru, RAW_TEST_FILE, tokenizer, label_map, maxlen=MAX_LEN, threshold=0.5)
        if df_bigru is not None:
            save_path = os.path.join(OUTPUT_DIR, "BIGRU_External_Predictions.csv")
            df_bigru.to_csv(save_path, index=False)
            print(f"📁 BIGRU predictions saved to: {save_path}")
        else:
            print("BIGRU model not found in memory.")
    else:
        print(f"⚠️ Could not find the file at: {RAW_TEST_FILE}")

...

>>> BILSTM EVALUATION <<<

--- ANALYZING RAW FASTA: TestExtLung.fasta ---
📁 Extracted 1 sequences from the raw file.
Running deep learning predictions (Threshold > 0.5)...
```

2. Skenario kedua (100 Augmentasi)

```

# Cell 12: Predict Directly from Raw External FASTA Files (A/B Test Version)
import os
import pickle
import numpy as np
import pandas as pd
from tensorflow.keras.preprocessing.sequence import pad_sequences

def predict_raw_fasta(model, fasta_path, tokenizer, label_mapping, maxlen, threshold=0.5):
    """Reads a raw .fasta file, predicts Cancer vs. Normal, and returns a DataFrame."""
    print(f"\n--- ANALYZING RAW FASTA: {os.path.basename(fasta_path)} ---")

    # 1. Read the raw FASTA file
    sequences = []
    seq_ids = []
    current_id, current_seq = None, []

    try:
        with open(fasta_path, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line: continue

                # Check for the FASTA header line
                if line.startswith(">"):
                    if current_id:
                        full_seq = "".join(current_seq).upper()
                        if len(full_seq) > 0:
                            sequences.append(full_seq[:maxlen])
                            seq_ids.append(current_id)
                        current_id = line[1:].split()[0]
                        current_seq = []
                    else:
                        current_seq.append(line)

                # Save the very last sequence in the file
                if current_id:
                    full_seq = "".join(current_seq).upper()
                    if len(full_seq) > 0:
                        sequences.append(full_seq[:maxlen])
                        seq_ids.append(current_id)

    except Exception as e:
        print(f"⚠️ Error reading file: {e}")
        return None

    if not sequences:
        print("⚠️ No valid sequences found in the file. Check the format.")
        return None

    print(f"📁 Extracted {len(sequences)} sequences from the raw file.")

    # 2. Tokenize and Pad (Using the specific Tokenizer passed into the function)
    X_ext = tokenizer.texts_to_sequences(sequences)

    # Using the specific maxlen passed into the function to avoid TF warnings!
    X_ext_padded = pad_sequences(X_ext, maxlen=maxlen, padding='post', truncating='post', value=0)

    # 3. Run the Model Predictions
    print(f"Running deep learning predictions (Threshold > {threshold})...\n")
    predictions_prob = model.predict(X_ext_padded, verbose=0)

    # 4. Map probabilities and compile results
    reverse_mapping = {v: k for k, v in label_mapping.items()}
    results_list = []

    for i, prob in enumerate(predictions_prob):
        prob_val = prob[0]

        predicted_class_idx = 1 if prob_val > threshold else 0
        predicted_class_name = reverse_mapping[predicted_class_idx]
        confidence = prob_val if predicted_class_idx == 1 else (1 - prob_val)

        results_list.append({
            'Sequence_ID': seq_ids[i],
            'Prediction': predicted_class_name,
            'Confidence(%)': round(confidence * 100, 2),
            'Raw_Probability': round(prob_val, 4)
        })

    if i < 10:
        print(f"🔗 Seq ID: {seq_ids[1]}")
        print(f"🔗 Prediction: {predicted_class_name} (Confidence: {confidence*100:.2f}%)")
```

```

except Exception as e:
    print(f"⚠️ Error reading file: {e}")
    return None

if not sequences:
    print("⚠️ No valid sequences found in the file. Check the format.")
    return None

print(f"📁 Extracted {len(sequences)} sequences from the raw file.")

# 2. Tokenize and Pad (Using the specific Tokenizer passed into the function)
X_ext = tokenizer.texts_to_sequences(sequences)

# Using the specific maxlen passed into the function to avoid TF warnings!
X_ext_padded = pad_sequences(X_ext, maxlen=maxlen, padding='post', truncating='post', value=0)

# 3. Run the Model Predictions
print(f"Running deep learning predictions (Threshold > {threshold})...\n")
predictions_prob = model.predict(X_ext_padded, verbose=0)

# 4. Map probabilities and compile results
reverse_mapping = {v: k for k, v in label_mapping.items()}
results_list = []

for i, prob in enumerate(predictions_prob):
    prob_val = prob[0]

    predicted_class_idx = 1 if prob_val > threshold else 0
    predicted_class_name = reverse_mapping[predicted_class_idx]
    confidence = prob_val if predicted_class_idx == 1 else (1 - prob_val)

    results_list.append({
        'Sequence_ID': seq_ids[i],
        'Prediction': predicted_class_name,
        'Confidence(%)': round(confidence * 100, 2),
        'Raw_Probability': round(prob_val, 4)
    })

if i < 10:
    print(f"🔗 Seq ID: {seq_ids[1]}")
    print(f"🔗 Prediction: {predicted_class_name} (Confidence: {confidence*100:.2f}%)")
```

```

    if i < 10:
        print(f"Seq ID: {seq_ids[i]}")
        print(f"Prediction: {predicted_class_name} (Confidence: {confidence*100:.2f}%)")
        print("-" * 50)

    if len(results_list) > 10:
        print(f"... and {len(results_list) - 10} more sequences processed.")

    return pd.DataFrame(results_list)

# =====
# HOW TO USE: THE 100-SEQUENCE A/B TEST
# =====
RAW_TEST_FILE = "/content/drive/MyDrive/DATA_SKRIPSI/testExtlung.fasta"
OUTPUT_DIR = "/content/drive/MyDrive/DATA_SKRIPSI/Predictions"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# --- 1. A/B TEST SETTINGS ---
# Point this to the specific folder where your 100-sequence data was saved
AUG_DIR_100 = "/content/drive/MyDrive/DATA_SKRIPSI/Augmented Data"

# ENTER THE EXACT MAX_LEN YOU WROTE DOWN DURING THE 100-SEQUENCE TRAINING:
# (If you forgot it, scroll up to the logs of Cell 5 & 6 for your 100-run and find "Dynamic MAX_LEN calculated: XXXX")
MAX_LEN_100 = 1250 # <--- CHANGE THIS NUMBER IF NECESSARY TO MATCH YOUR LOGS!

# --- 2. LOAD SPECIFIC ARTIFACTS ---
print("\n--- LOADING 100-SEQUENCE ARTIFACTS ---")
try:
    with open(os.path.join(AUG_DIR_100, 'dna_tokenizer.pickle'), 'rb') as handle:
        tokenizer_100 = pickle.load(handle)
    with open(os.path.join(AUG_DIR_100, 'label_map.pickle'), 'rb') as handle:
        label_map_100 = pickle.load(handle)
    print("✅ Successfully loaded 100-Sequence Tokenizer and Label Map.")
except Exception as e:
    print(f"❌ Error loading artifacts. Did you save them in (AUG_DIR_100)?")
    print(f"Error details: {e}")
    tokenizer_100, label_map_100 = None, None

# --- 3. RUN THE TEST ---
if os.path.exists(RAW_TEST_FILE) and tokenizer_100 is not None:

```

```

# --- 3. RUN THE TEST ---
if os.path.exists(RAW_TEST_FILE) and tokenizer_100 is not None:

    # --- BILSTM Evaluation ---
    print("\n" + "-"*50)
    print("    >>> 100-SEQUENCE BILSTM EVALUATION <<<")
    print("-"*50)
    if 'model_bilstm' in locals():
        df_bilstm = predict_raw_fasta(model_bilstm, RAW_TEST_FILE, tokenizer_100, label_map_100, maxlen=MAX_LEN_100, threshold=0.5)
        if df_bilstm is not None:
            save_path = os.path.join(OUTPUT_DIR, 'BILSTM_100Seq_Predictions.csv')
            df_bilstm.to_csv(save_path, index=False)
            print(f"✅ BILSTM predictions saved to: {save_path}")
        else:
            print("BILSTM model not found in memory.")

    # --- BIGRU Evaluation ---
    print("\n" + "-"*50)
    print("    >>> 100-SEQUENCE BIGRU EVALUATION <<<")
    print("-"*50)
    if 'model_bigru' in locals():
        df_bigru = predict_raw_fasta(model_bigru, RAW_TEST_FILE, tokenizer_100, label_map_100, maxlen=MAX_LEN_100, threshold=0.5)
        if df_bigru is not None:
            save_path = os.path.join(OUTPUT_DIR, 'BIGRU_100Seq_Predictions.csv')
            df_bigru.to_csv(save_path, index=False)
            print(f"✅ BIGRU predictions saved to: {save_path}")
        else:
            print("BIGRU model not found in memory.")

    else:
        print(f"⚠️ Test aborted. Check if (RAW_TEST_FILE) exists and your tokenizer loaded correctly.")

...

--- LOADING 100-SEQUENCE ARTIFACTS ---
✅ Successfully loaded 100-Sequence Tokenizer and Label Map.

=====
>>> 100-SEQUENCE BILSTM EVALUATION <<<

```

LAMPIRAN 7. BIODATA PENULIS

BIODATA

Nama : Rendika Rahmaturrizki
Tempat, tanggal lahir : Jakarta, 24 Mei 2002
Alamat : Jl. Bak Lilip, Lamgapang, Ulee Kareng, Aceh Besar
Nama Ayah : Alm. Yulizar
Pekerjaan Ayah : Wiraswasta
Nama Ibu : Monalisa
Pekerjaan Ibu : Ibu Rumah Tangga
Alamat Orang Tua : Jl. Simpang Gunung Cut, Tengah Baru, Labuhan Haji, Aceh Selatan

Riwayat Pendidikan :

Jenjang	Nama Sekolah	Bidang Studi	Tempat	Tahun Ijazah
SD	SD Lubang Buaya 12 Pagi	-	Jakarta Timur	2014
SMP	SMPN 81 Jakarta	-	Jakarta Timur	2017
SMA	SMAN 61 Jakarta	IPA	Jakarta Timur	2020

Banda Aceh, 19 Juni 2026

Rendika Rahmaturrizki
NPM. 2108107010066